



University
of Basel

Chapter 5.1: **Parallelization Abstractions: Steps** **Foundations of Distributed Systems**

Fall Semester 2025

Prof. Dr. Florina Ciorba



Addressed in The **Processing** Lecture Series

Chapter 5.1: Parallelization abstractions



Performance

```
00111101001010101
01001010100010101
01010ERROR011010
10100010100110001
01100010100100111
10100010100110001
```

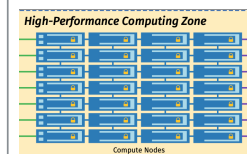
Resiliency



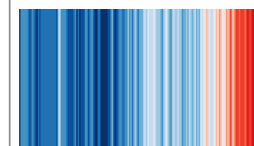
**Extreme
scalability**



Energy

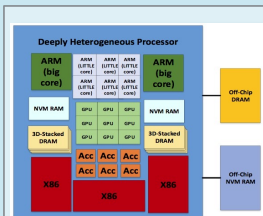


Security

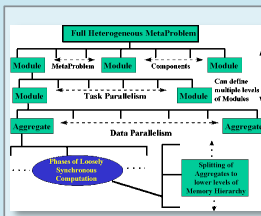


Sustainability

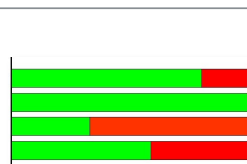
Chapter 5.3: Parallel programming



Heterogeneity



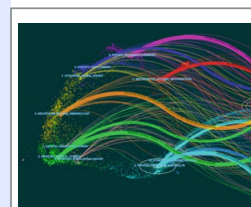
**Multilevel
parallelism**



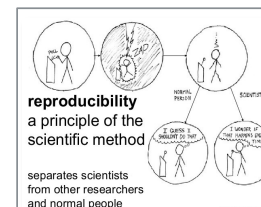
Load imbalance



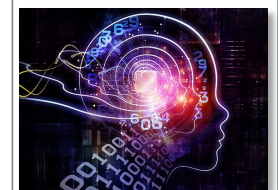
Programming



Visualization



Reproducibility



**Convergence
HPC+Data+AI**

```
00111101001010101
01001010100010101
01010ERROR01101010
10100010100110001
01100010100100111
10100010100110001
```



The diagram illustrates a 'High-Performance Computing Zone' containing a 4x4 grid of 16 'Compute Nodes'. Each node is represented by a blue rectangle with a yellow border, divided into four quadrants. The top-left quadrant contains a small icon of a server rack, the top-right contains a small icon of a network switch, the bottom-left contains a small icon of a storage unit, and the bottom-right contains a small icon of a monitor. The nodes are interconnected by a network of lines, with a central horizontal line and vertical lines connecting the nodes in a grid pattern. The entire zone is enclosed in a dashed yellow border.

Age Group	Percentage
18-24	22%
25-34	18%
35-44	15%
45-54	12%
55-64	10%
65-74	8%
75-84	5%
85+	3%

The diagram illustrates a deeply heterogeneous processor architecture. It features two main processing units: an ARM (big core) on the left and an ARM (arm core) on the right. Each core has its own NVM RAM and 3D-Stacked DRAM. The ARM (big core) also includes a cache hierarchy with L1, L2, and L3 levels. The ARM (arm core) includes a cache hierarchy with L1 and L2 levels. Both cores are connected to a shared system bus. The system bus is connected to an Off-Chip NVM RAM and an Off-Chip 3D-Stacked DRAM. The architecture is designed for high performance and low power consumption.

The diagram illustrates the decomposition of a 'Full Heterogeneous MetaProblem' into 'Module' and 'Component' blocks. This leads to 'Task Parallelism' and 'Data Parallelism'. It also shows 'Phases of Locality Synchronous Computation' and 'Stripping of Aggregates to lower levels of Memory Hierarchy'.

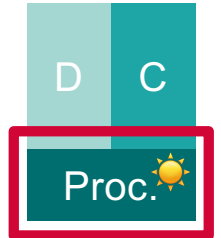
Responsibility	Percentage
Current government	75%
Opposition	25%

reproducibility
a principle of the scientific method

separates scientists
from other researchers
and normal people

1-3

Exercises 5 and 6 (HPC Part)



Exercises	Hand out Date	Hand in Deadline	Interviews Ex 5+6
5 (OpenMP)	Nov. 06, 2025 (week 8)	Nov. 16, 2025 (week 10)	Nov. 24-28, 2025 (week 11)
6 (MPI)	Nov. 13, 2025 (week 9)	Nov. 23, 2025 (week 11)	

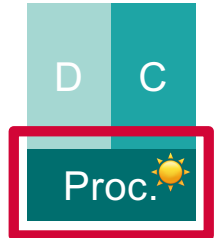
Overview of Chapter 5.1: Parallelization Abstractions

- a Parallelization Basic Abstractions
 - b Parallelism: Forms, Levels, Machine Models, Types
-

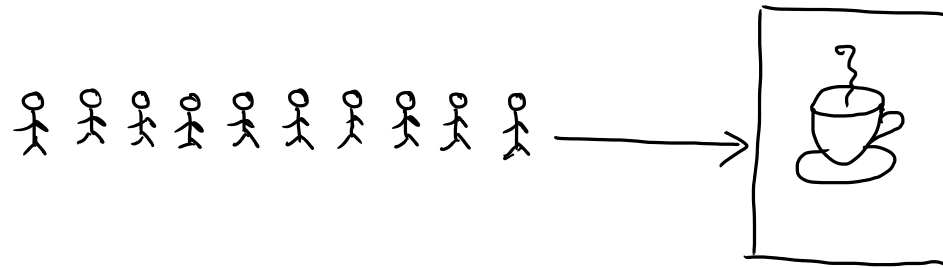
Learning objectives

Understand the basic abstractions and principles for parallelizing an application.
Understand the forms, levels, types and taxonomy of parallelism.
Consider these in context.

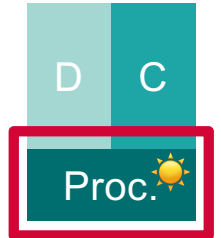
Sequential Computing



one queue → *one* coffee machine

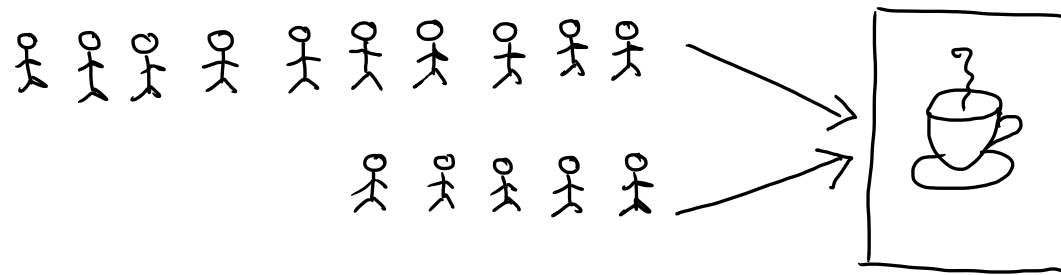


Concurrent Computing

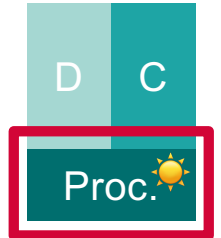


two queues → *one* coffee machine

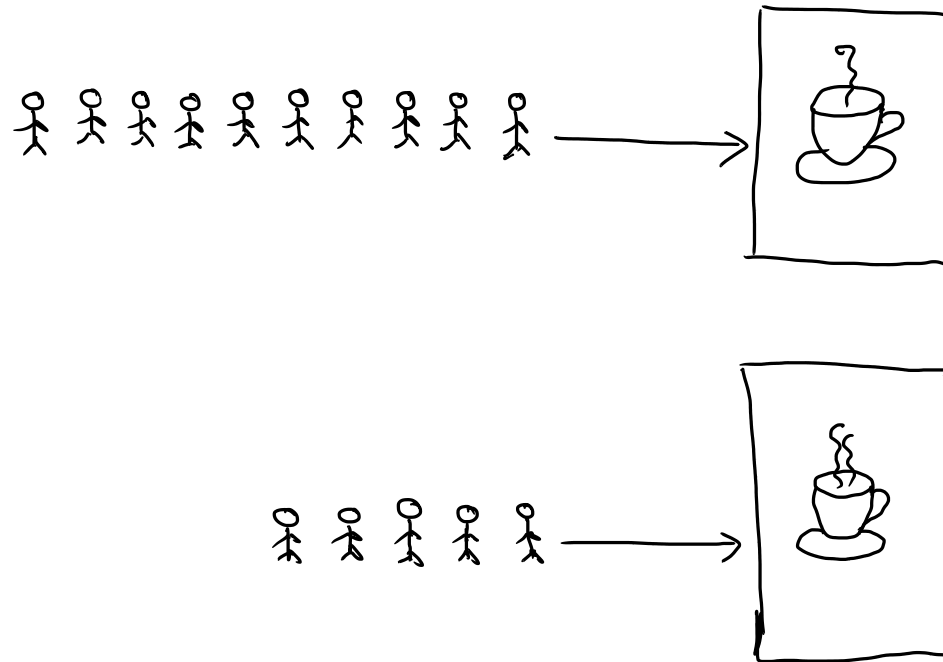
Concurrency = Two Queues, One Coffee Machine



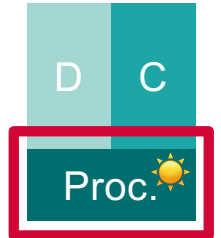
Parallel Computing



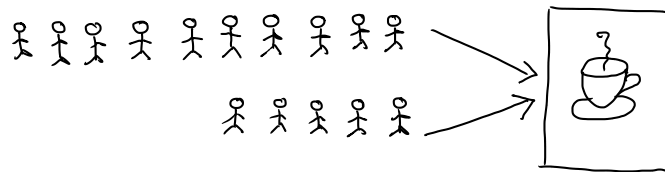
two queues $\rightarrow$ *two* coffee machines
Parallelism = Two Queues, *Two* Coffee Machines



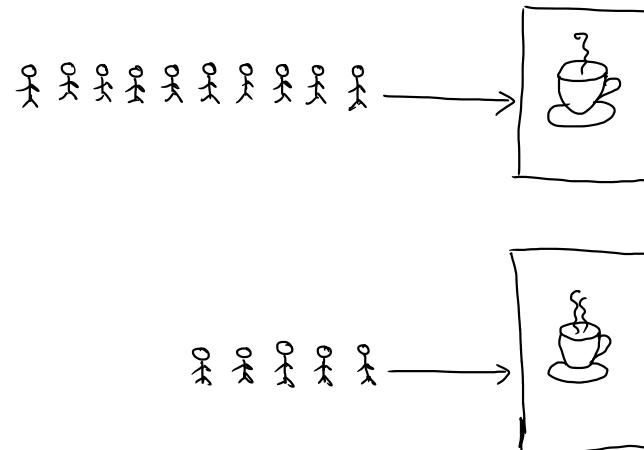
Concurrency vs. Parallelism



two queues → *one* coffee machine

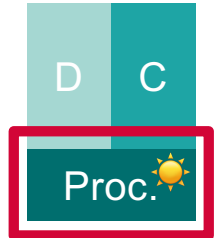


two queues → *two* coffee machines

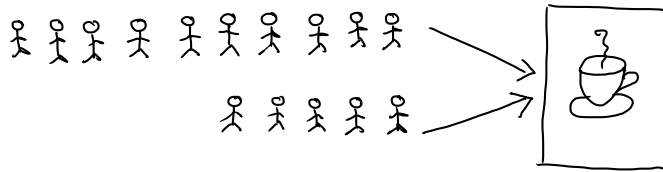


Concurrency ≠ Parallelism

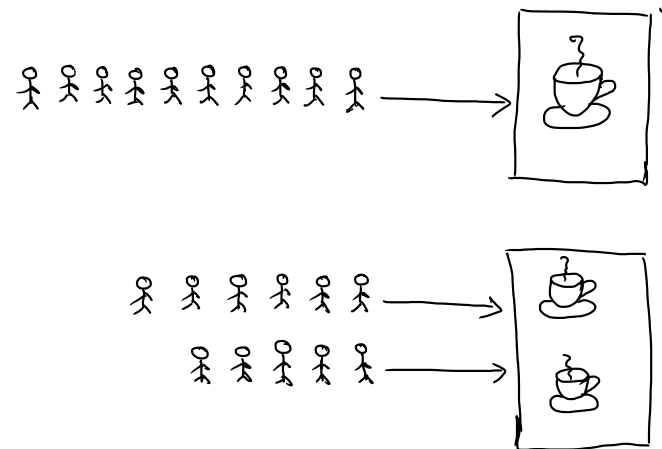
Concurrency vs. Parallelism



two queues → *one* coffee machine



three queues → *two* coffee machines
(one with *dual*-coffee support)



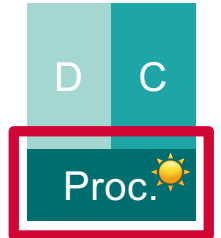
Concurrency: **problem** property.

Parallelism: **machine** property.

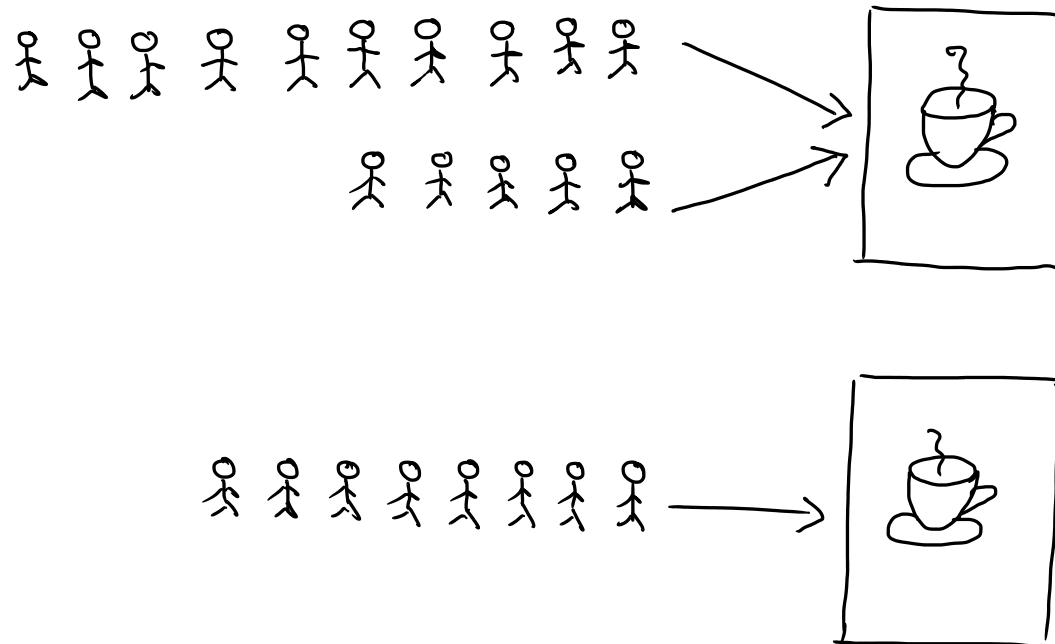
Sequential: **machine** property (special case of parallelism, with a single machine)

Concurrency ≠ Parallelism

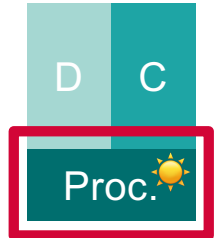
Concurrency + Parallelism



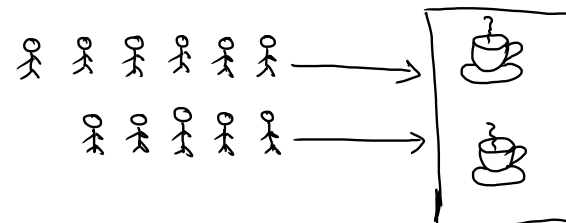
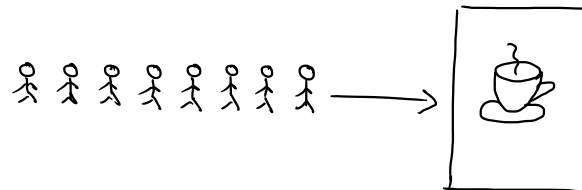
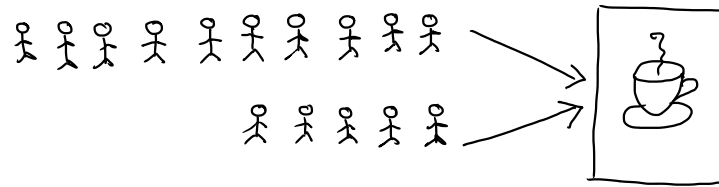
three queues → *two* coffee machines



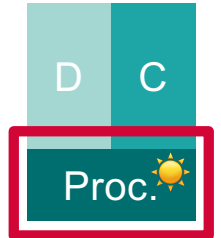
Concurrency + Parallelism



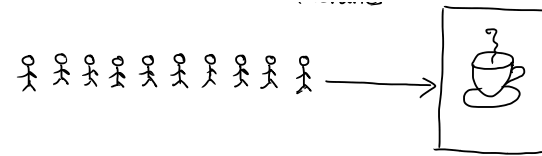
five queues → *three* coffee machines (one with *dual*-coffee support)



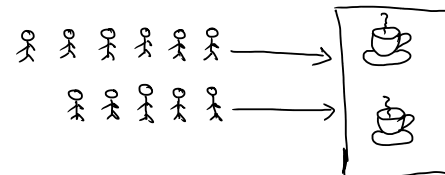
Parallelism + Distribution



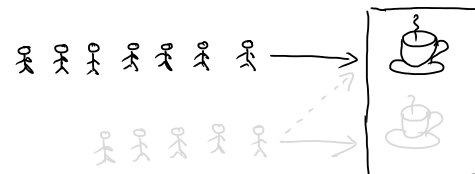
multiple queues → *several distributed* coffee machines (some with *dual*-coffee support)



Parallelism: a (large) *efficient* team of *close* resources.

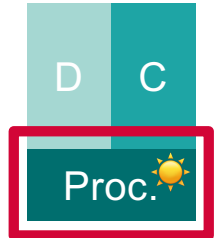


Distribution: a *large* and *efficient* team of not (necessarily) close resources.



Parallelism ≠ Distribution

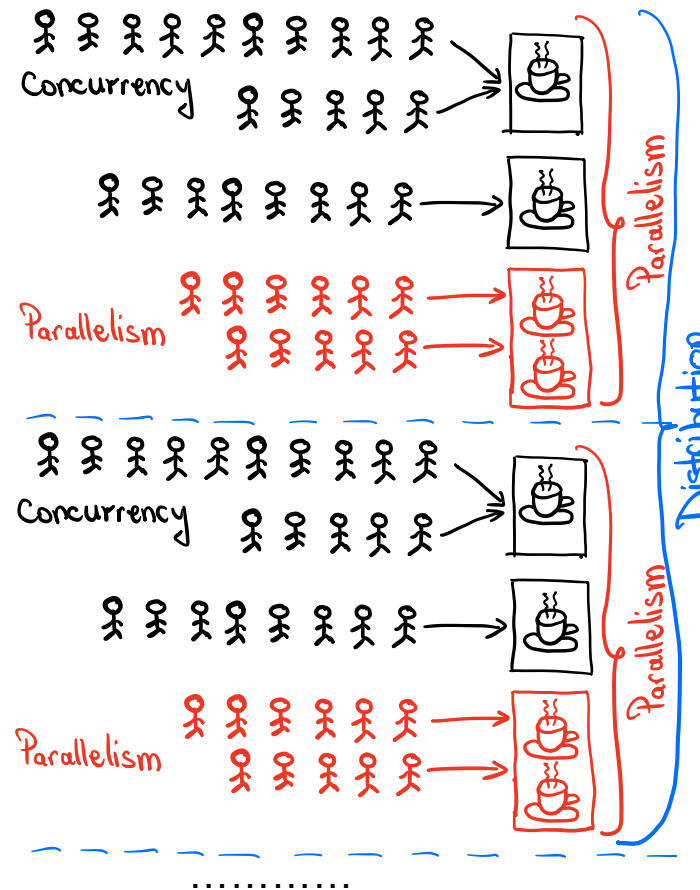
HPC = Concurrency + Parallelism + Distribution in Processing of Computations



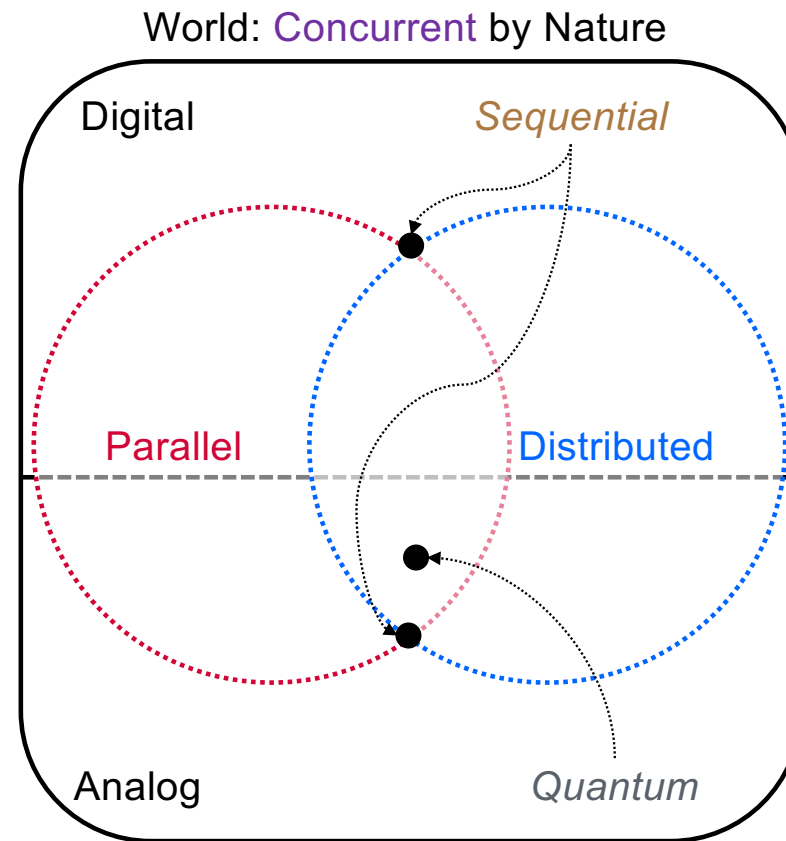
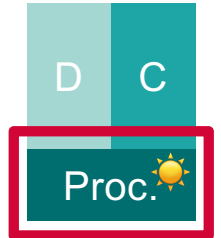
coffee queue

👁 issues?

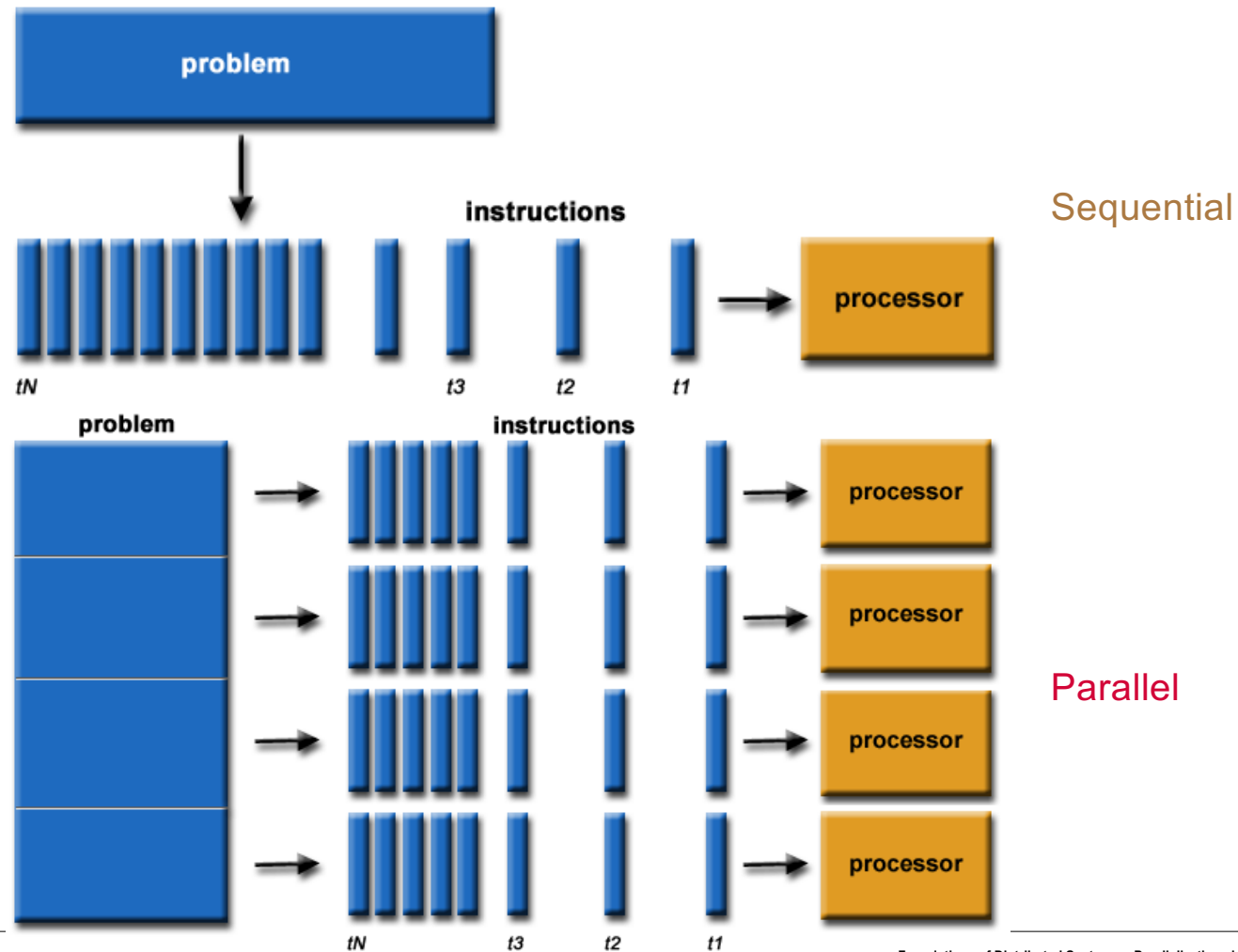
tea queue



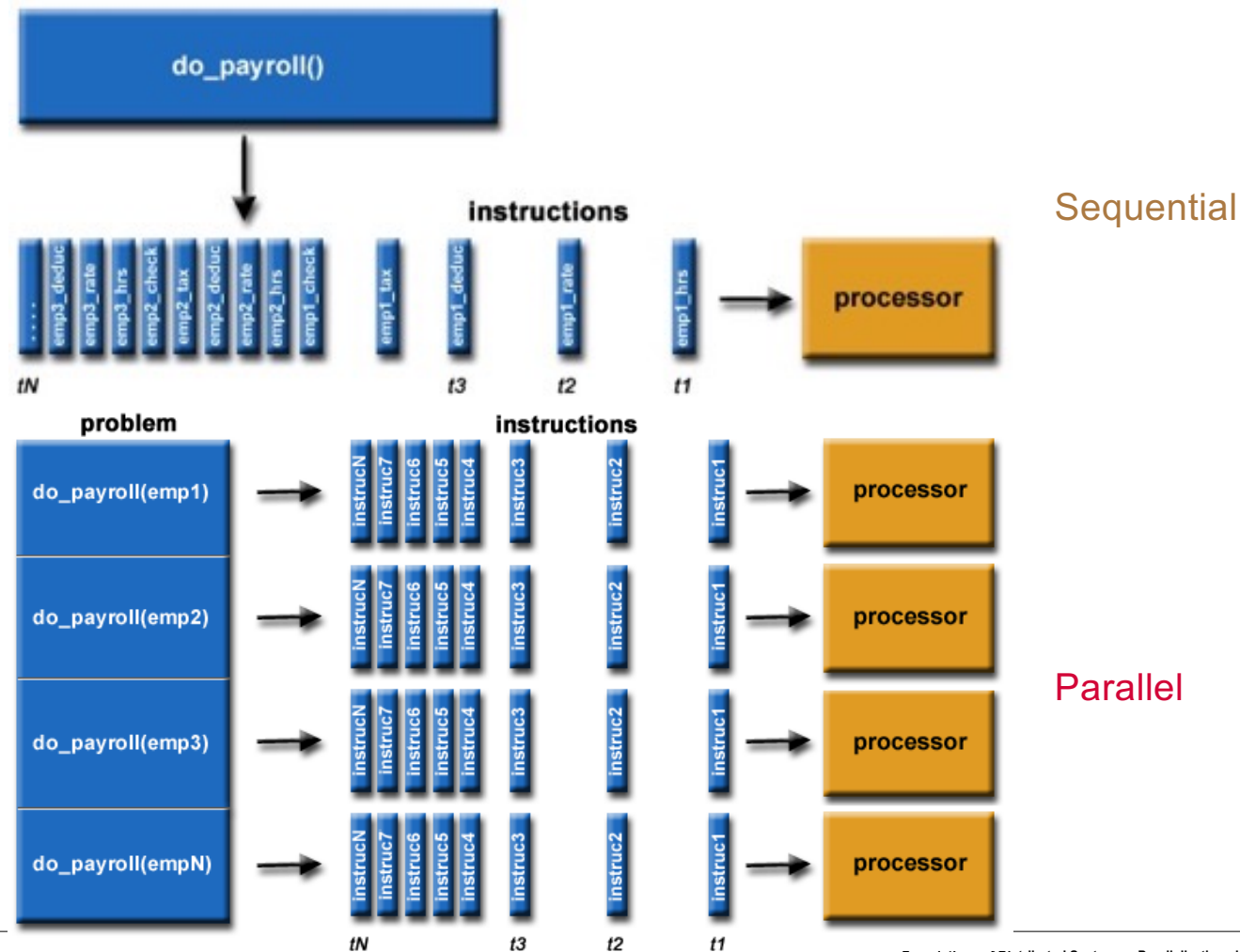
Parallel and Distributed **Computing** Systems in Context*



Sequential vs. Parallel Computing (1)



Sequential vs. Parallel Computing (2)



Sequential Program = Sequential Steps + Programming

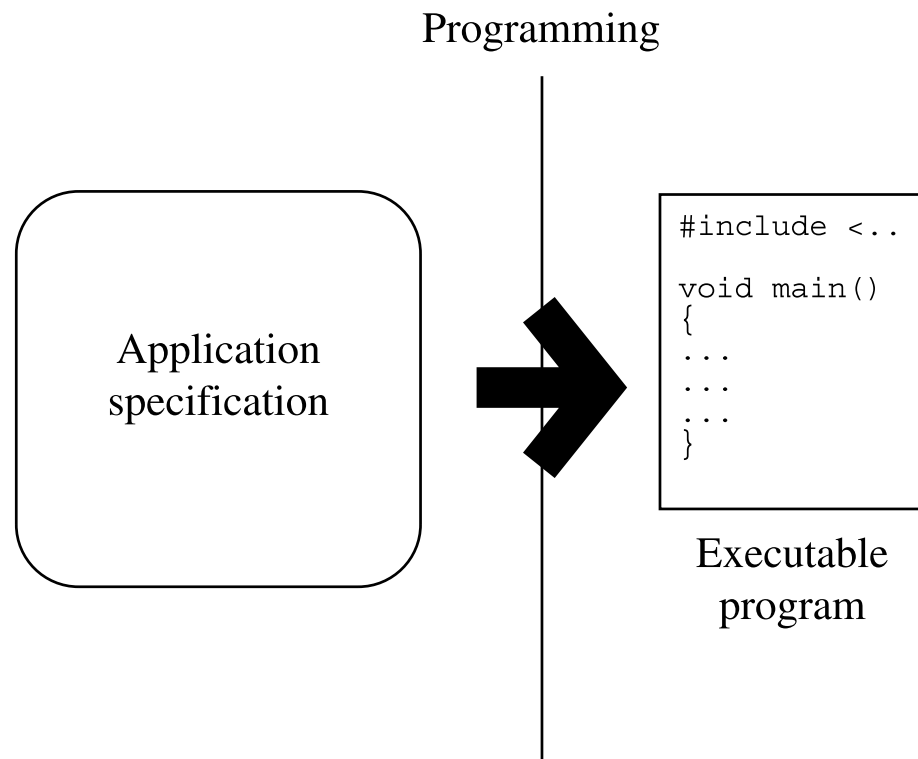
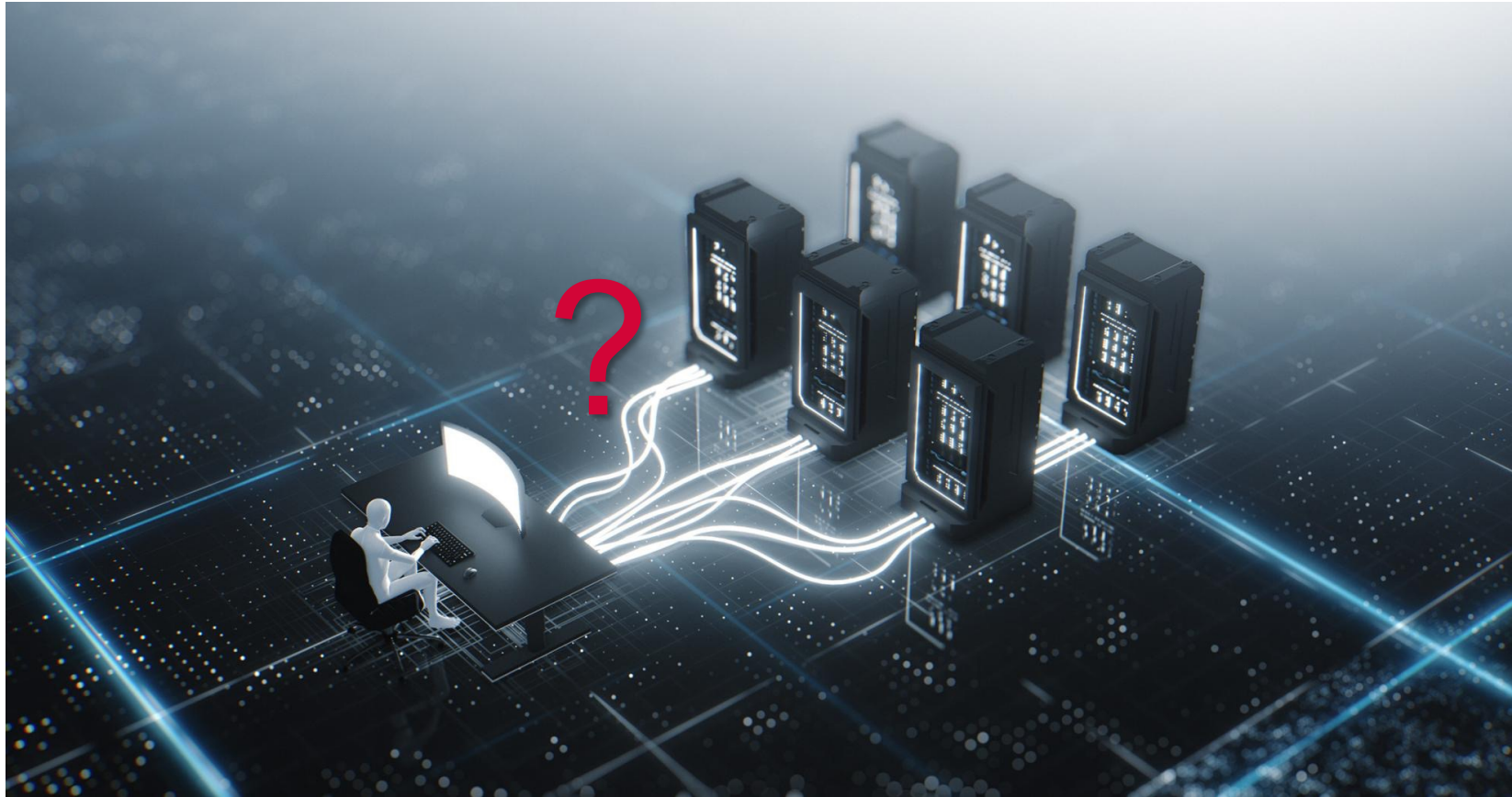


Figure 2.14. Sequential programming.

Parallel Program = Parallelization + Programming



© image by photographer or artist - Getty / copyright holder

Parallel Program = Parallelization + Programming

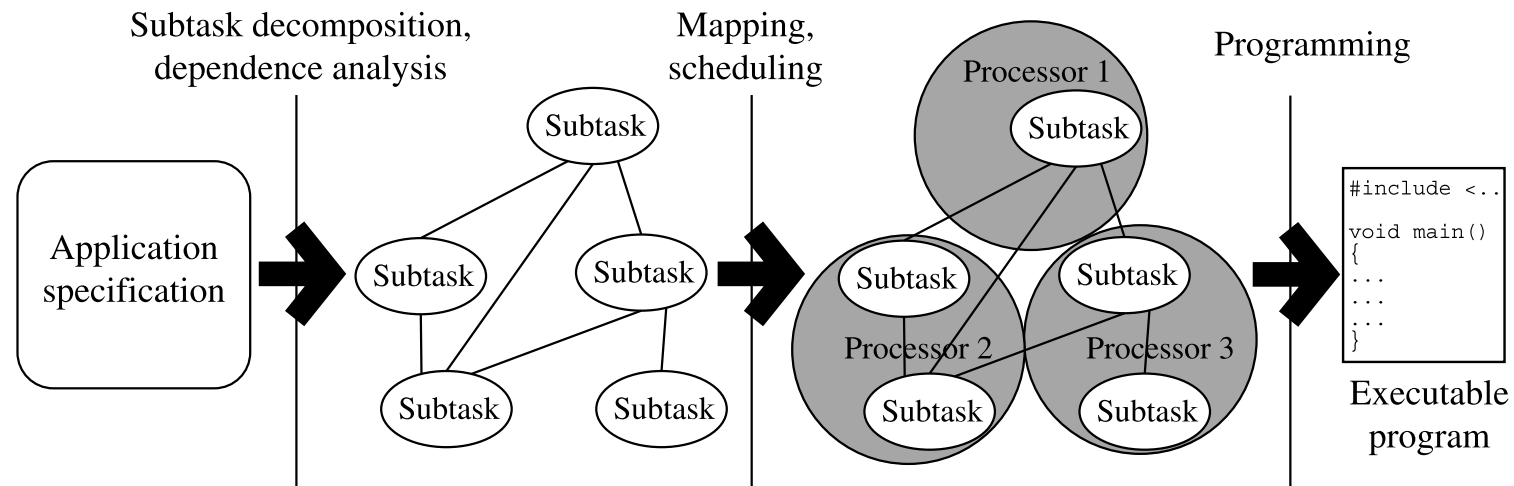


Figure 1.1. Parallel programming—process of parallelization.

Parallelization (1)

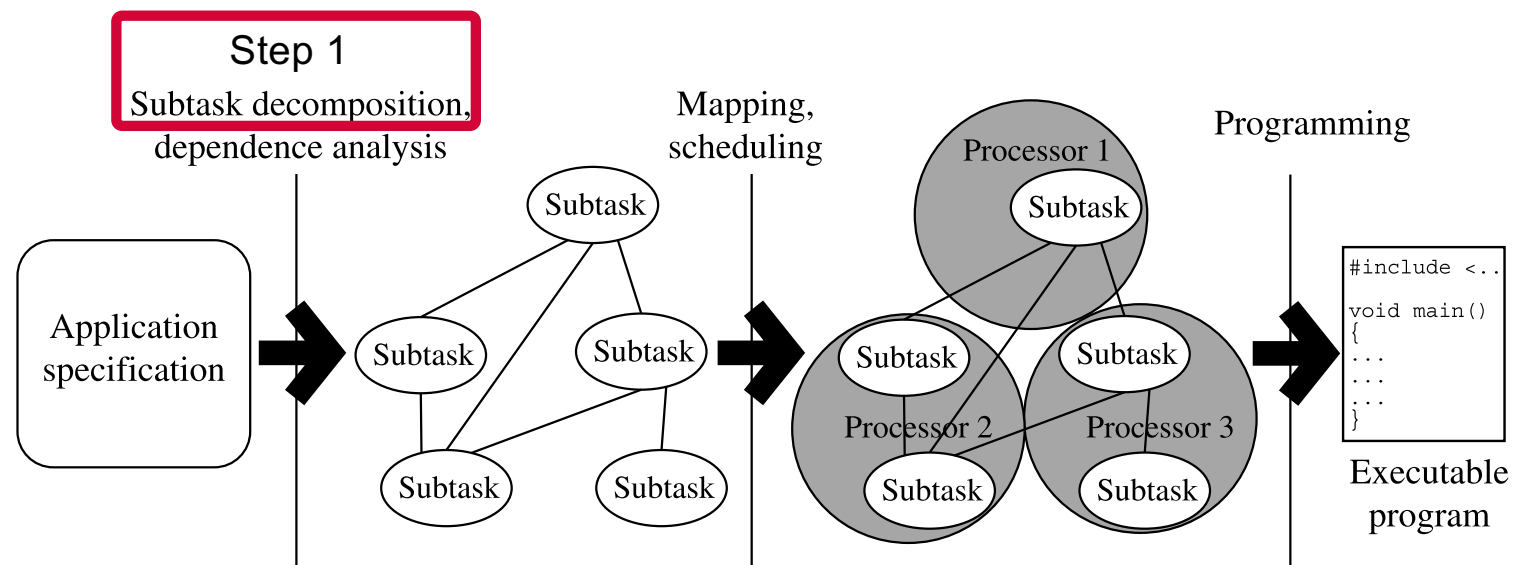


Figure 1.1. Parallel programming—process of parallelization.

Step 1: **Decomposition** of applications into functional tasks / data blocks

Ideally: exposes the inherent concurrency of an application.

Parallelization (2)

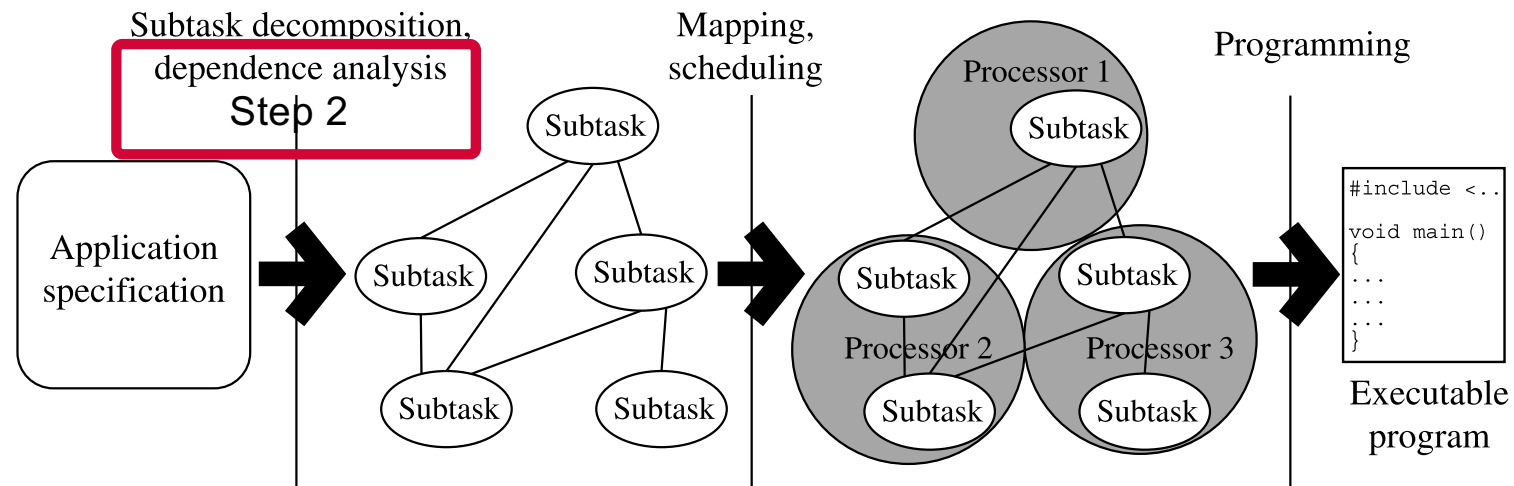


Figure 1.1. Parallel programming—process of parallelization.

Step 2: **Dependence analysis** of (any) dependencies between the decomposed

- * functional tasks (control dependencies) OR
- * data blocks (data dependencies)

Parallelization (3)

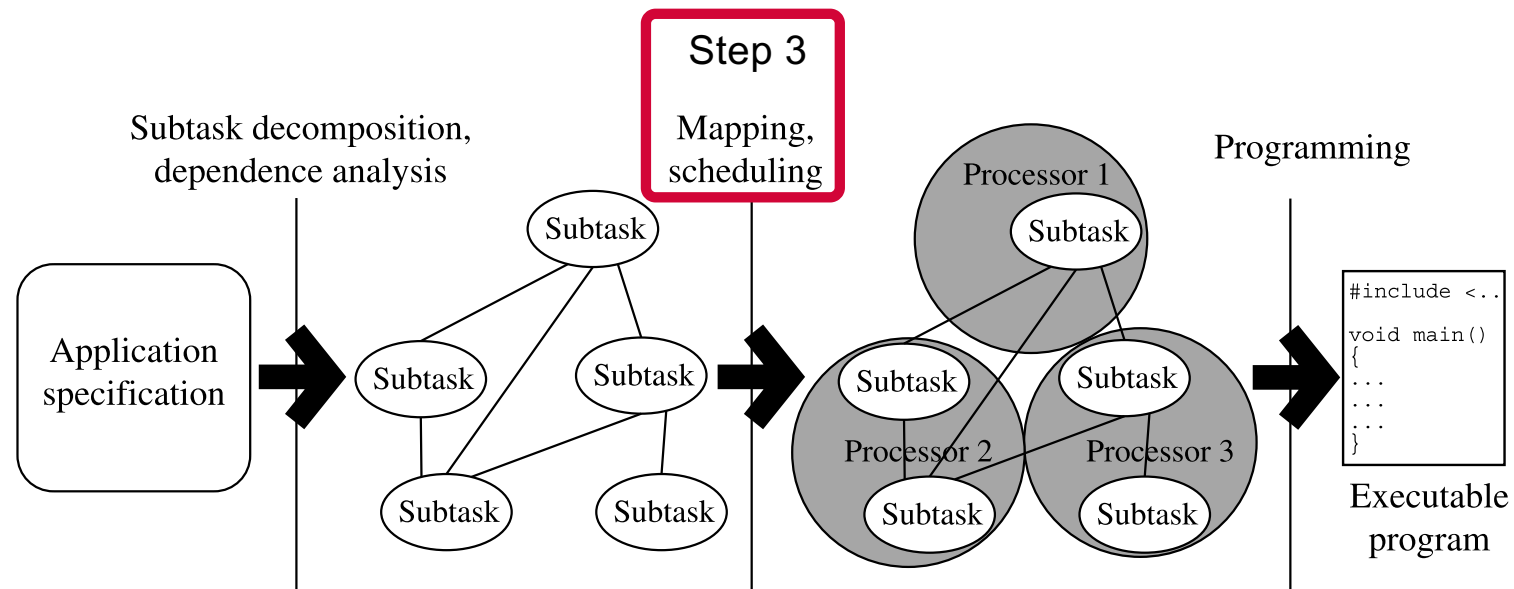


Figure 1.1. Parallel programming—process of parallelization.

Step 3: **Mapping** (also assignment, part of scheduling) of the execution of functional tasks / operations on data blocks onto the target parallel or distributed computing system while satisfying the dependencies (if any)

Parallelization + Parallel Programming (4)

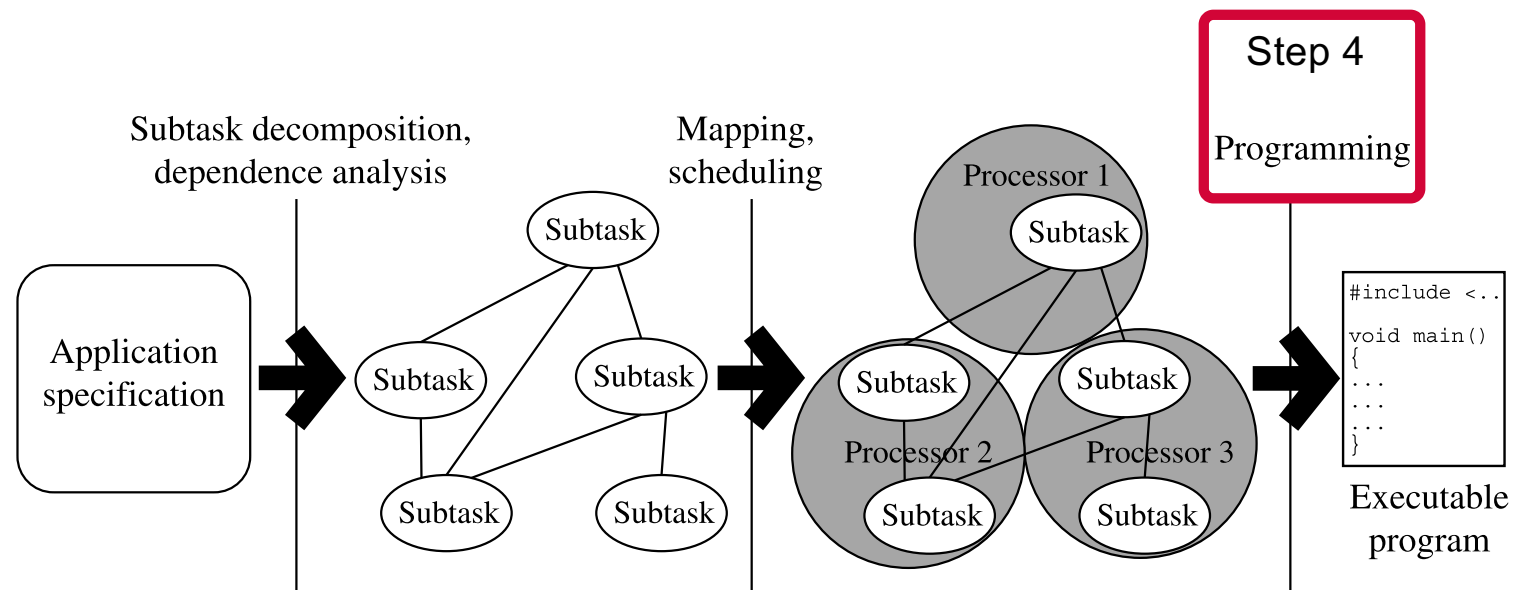


Figure 1.1. Parallel programming—process of parallelization.

Step 4: **Programming** or expressing all the parallelism exposed in Steps 1-3 via a programming language towards an executable parallel program

Parallelization + Parallel Programming

Analysis of *large* or *heavy* **sequential** programs to **identify parallelism** and to restructure them for the **purpose of executing them efficiently** on **parallel** and/or **distributed** computing systems

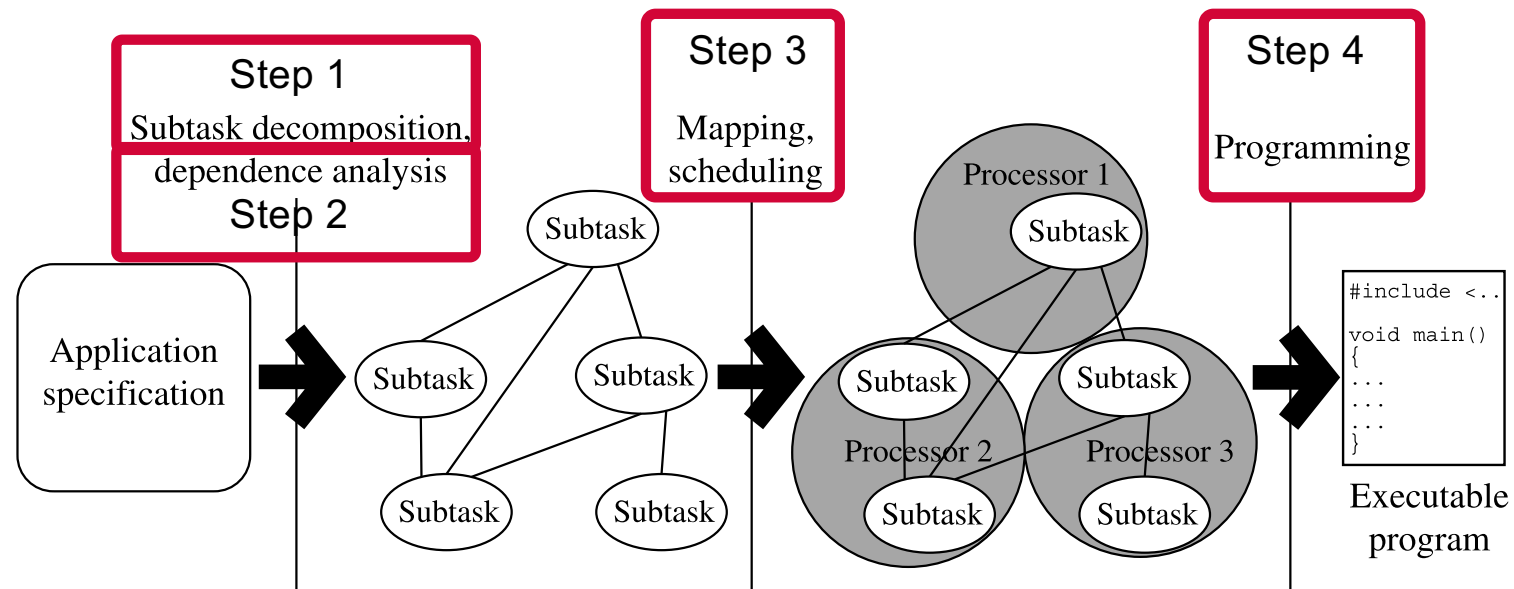


Figure 1.1. Parallel programming—process of parallelization.

Parallelization + Parallel Programming

Analysis of *large* or *heavy* **sequential** programs to **identify parallelism** and to restructure them for the **purpose of executing them efficiently** on **parallel** and/or **distributed** computing systems

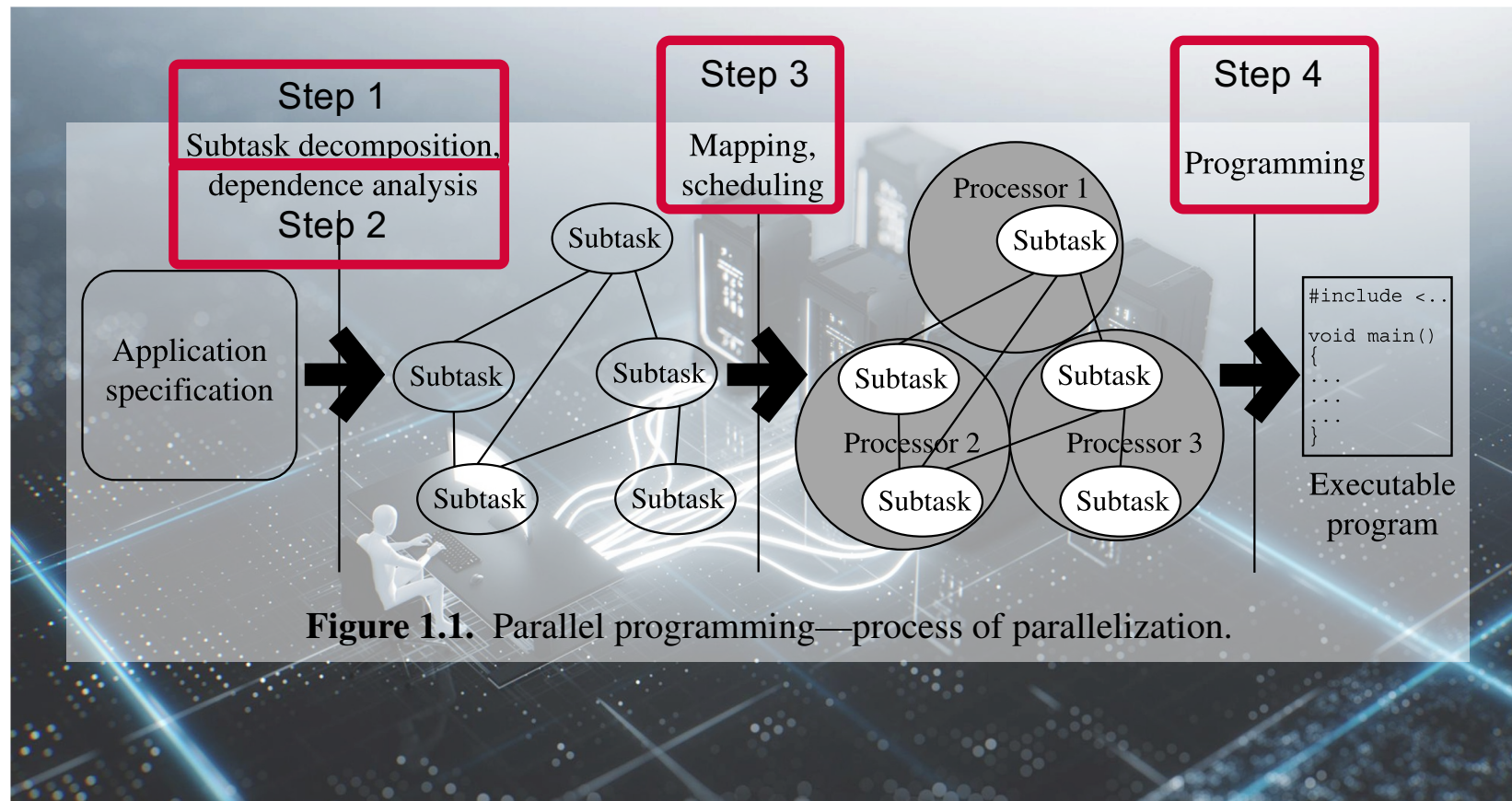
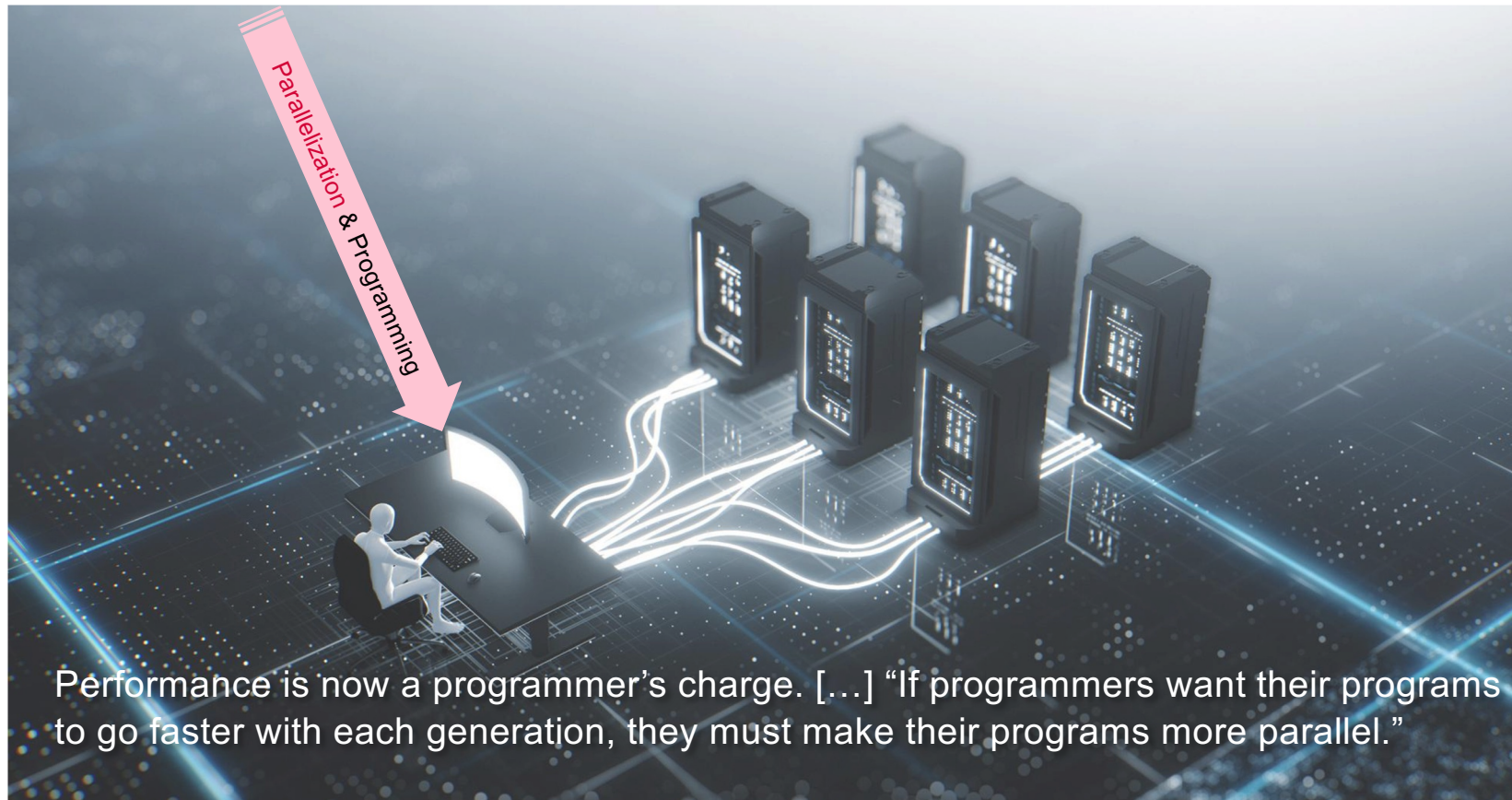


Figure 1.1. Parallel programming—process of parallelization.

Parallelization + Parallel Programming

Analysis of *large* or *heavy* **sequential** programs to **identify parallelism** and to restructure them for the **purpose of executing them efficiently** on **parallel** and/or **distributed** computing systems



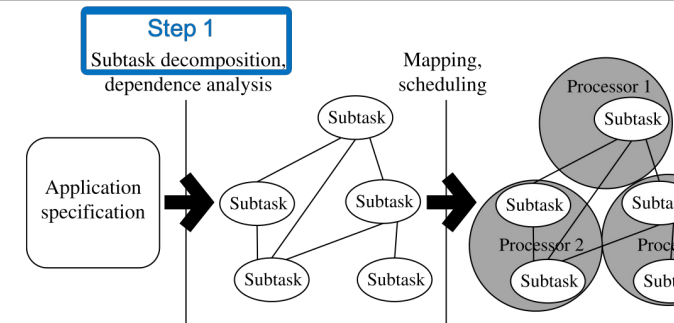
Parallelization Step 1: Decomposition

Decomposition is influenced by the following **factors**

a. **Application type** – **distinct steps** or one **iterative** block of (ir/regular) **computations**

Data parallel (problem data set divided into subsets for the same tasks)

Task parallel (problem consists of tasks/steps of different types)



Parallelization Step 1: Decomposition

Decomposition is influenced by the following **factors**

a. **Application type** – distinct steps or one iterative block of (ir/regular) computations

b. **Concurrency degree (max, avg)** – depending on the degree of **inherent** parallelism, applications can be

Embarrassingly parallel

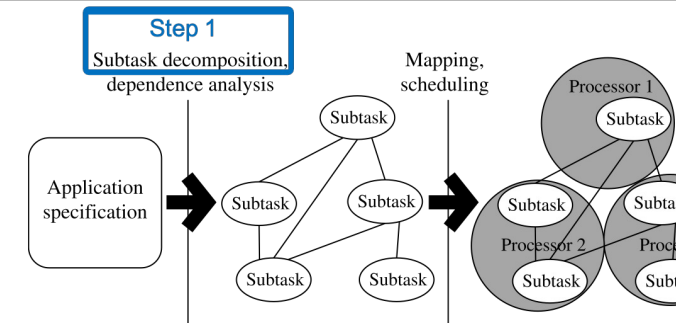
(**all** tasks can be executed *concurrently*)

Pleasingly parallel

(enough tasks can be executed *concurrently*)

Embarrassingly serial

(**no two tasks** can be executed *concurrently*)



Parallelization Step 1: Decomposition

Decomposition is influenced by the following **factors**

a. **Application type** – distinct steps or one iterative block of (ir/regular) computations

b. **Concurrency degree (max, avg)** – applications can be

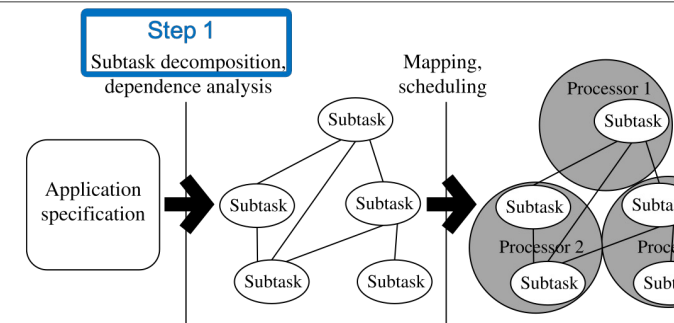
Embarrassingly parallel (***all*** tasks can be executed *concurrently*)

Pleasingly parallel (enough tasks can be executed *concurrently*)

Embarrassingly serial (***no two tasks*** can be executed *concurrently*)

c. **Granularity** – denotes the computational **size** of tasks or **size** of data blocks after the decomposition

Fine, **medium** and **coarse**; derives from above **inherent** degree of concurrency



Parallelization Step 1: Decomposition

Decomposition is influenced by the following **factors**

a. Application type – distinct steps or one iterative block of (ir/regular) computations

b. Concurrency degree (max, avg) – applications can be

Embarrassingly parallel (***all*** tasks can be executed *concurrently*)

Pleasingly parallel (enough tasks can be executed *concurrently*)

Embarrassingly serial (***no two tasks*** can be executed *concurrently*)

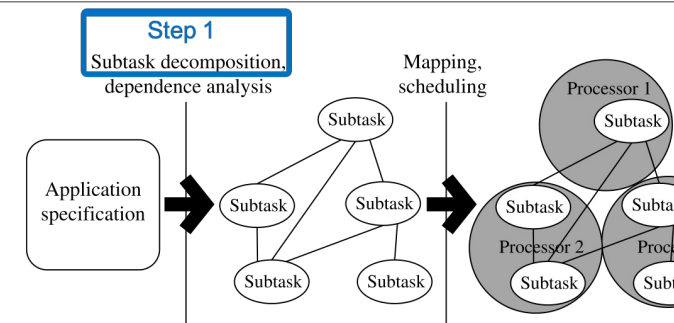
c. Granularity – denotes the computational size of tasks or size of data blocks after the decomposition

Fine, medium and coarse; derives from above inherent degree of concurrency

d. Target system – affects costs of *synchronization* and *communication*

Shared-memory architectures usually support a *fine-grain* decomposition (*inexpensive* more frequent synchronization and communication)

Distributed-memory architectures usually require a *coarse-grain* decomposition (*more expensive* less frequent synchronization and communication)



Parallelization Step 1: Decomposition Types

There are several **types** of decomposition

Data decomposition – exploits **data** parallelism

Functional decomposition – exploits **functional** (task) parallelism

Recursive decomposition – divide-and-conquer approach, exploits **data** and/or **functional** parallelism

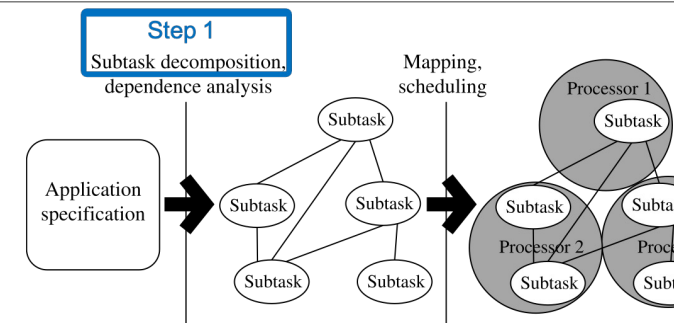
Exploratory decomposition – branch-and-bound approach, search problems, exploits **data** and/or **functional** parallelism

Speculatory decomposition – probabilistic approach, exploits and employs 'if' statements, exploits **data** and/or **functional** parallelism

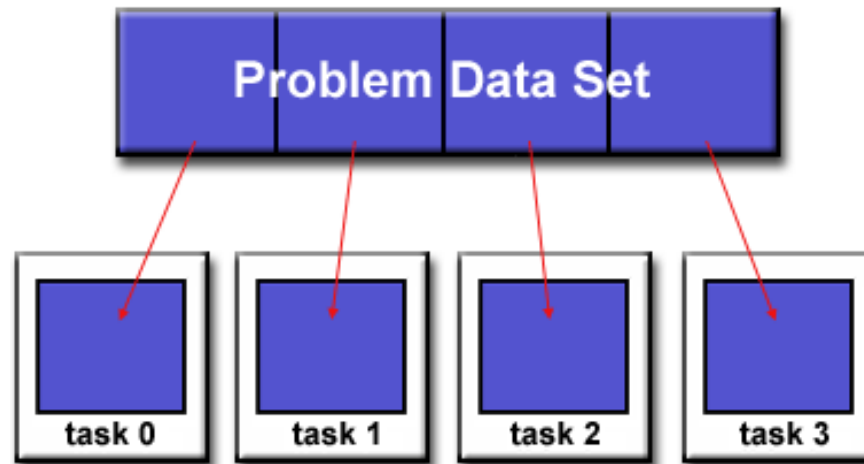
⚠ These are not strictly distinct and can be combined.

Example: searching for a name in a large collection of names can employ

- Data decomposition of the names dataset and
- Exploratory decomposition for searching the specific name



Parallelization Step 1: Decomposition Type Data



Type: *data decomposition*

- Data: problem input dataset divided into pieces
 - More regular data set => more massive parallelization
- Input: problem data set, #tasks (degree of concurrency, here 4), deps (here none), hw par. (4)
- **Output:** 4 data subsets

Applications: all **data parallel** problems (computational sciences, big data, AI/ML/DL eg batch parallelism)

Data: image is divided into chunks for concurrent execution

Execution: Each pixel/pixel batch processed independently

Parallelization Step 1: Decomposition Type/Example Data

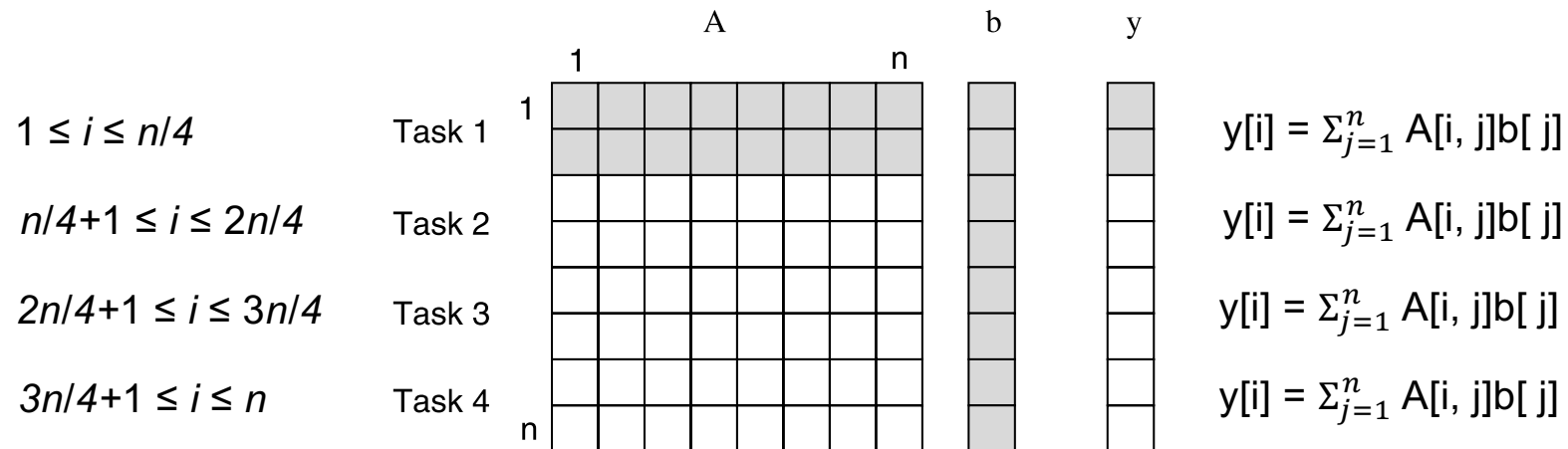
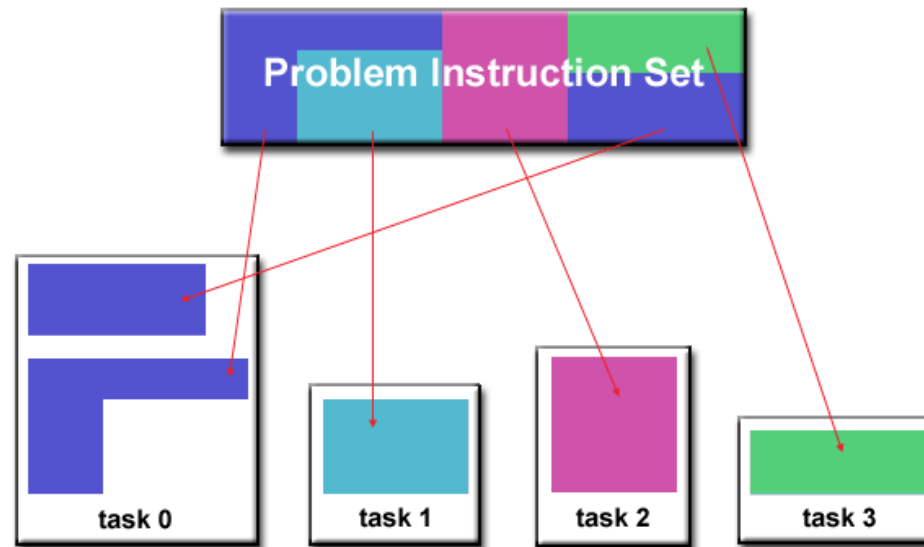


Figure 2.16. Data composition of matrix–vector multiplication $A \cdot b = y$. Data is partitioned into $N = 4$ parts corresponding to 4 tasks; the gray-shaded part shows the data accessed by task 1 (Grama et al. [82]).

Type: *data decomposition* for an image processing problem

- Data: input (A, b), output (y) [, intermediate]
- Input: data (A; b), #tasks (degree of concurrency, here 4), dependencies (here none), hw parallelism (here 4)
- Output: A and y partitioned into subsets (here 4), b replicated

Parallelization Step 1: Decomposition Type Functional



Type: *functional decomposition*

- Tasks: execute different instructions of the parallel program
- Input: problem instruction set, **#tasks** (degree of concurrency, here 4), **dependencies** (here none), **hw parallelism** (here 4)
- Output: **partitioned** task subsets (here 4)

Applications: all **task parallel** problems, all **graph** problems, such as graphics pipelines, data streaming, multimedia processing

Parallelization Step 1: Decomposition Type/Example Functional

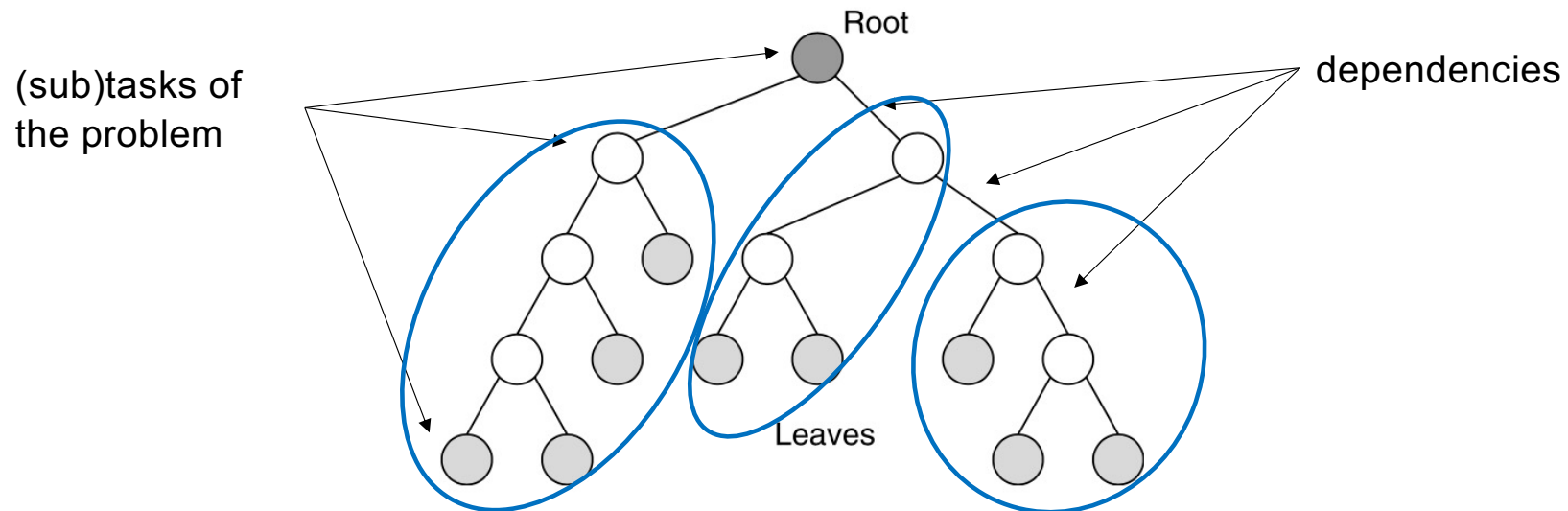
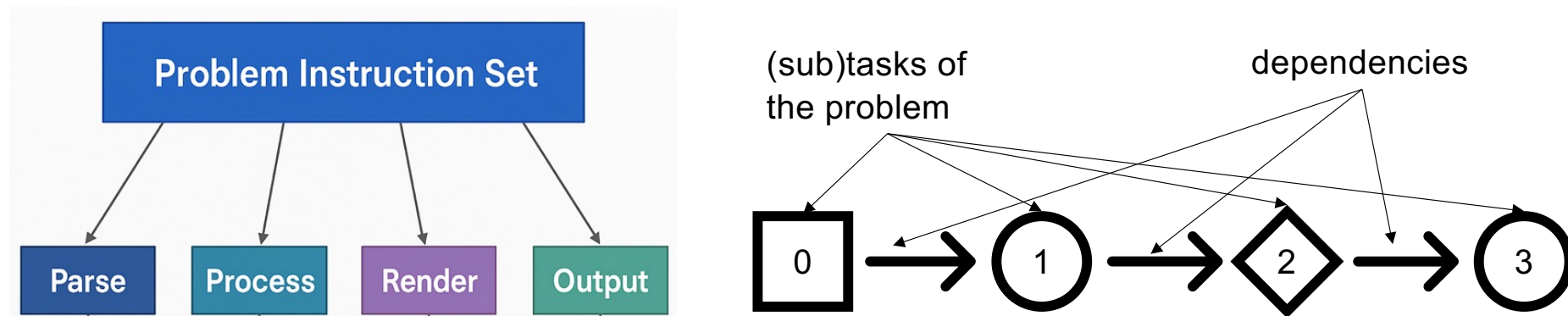


Figure 2.17. Tree created by a **functional** decomposition.

Type: *functional decomposition* of a program

- Tasks: execute different instructions in the program
- Input: program, **#tasks (here 17)**, **dependencies (here #edges)**, hw parallelism (here 3)
- Output: **partitioned** task sets (here 3 with 7, 4, and 5 tasks resp.)
- Execution: Each task set can run in parallel (*after step 3 mapping*)

Parallelization Step 1: Decomposition Type/Example Functional



Type: *functional decomposition* of a Web Service Pipeline

- Tasks: 0 Parse & validate input, 1 Compute response, 2 Render response, 3 Output response
- Input: program, #tasks (here 4), dependencies (here 3), hw parallelism (here 4)
- Output: *partitioned* task sets (here 4 task sets, each with 1 task)
- Execution: Each function can run in parallel (*dependencies step 2 => pipeline style*) for different requests

Parallelization Step 1: Decomposition Type/Example Recursive

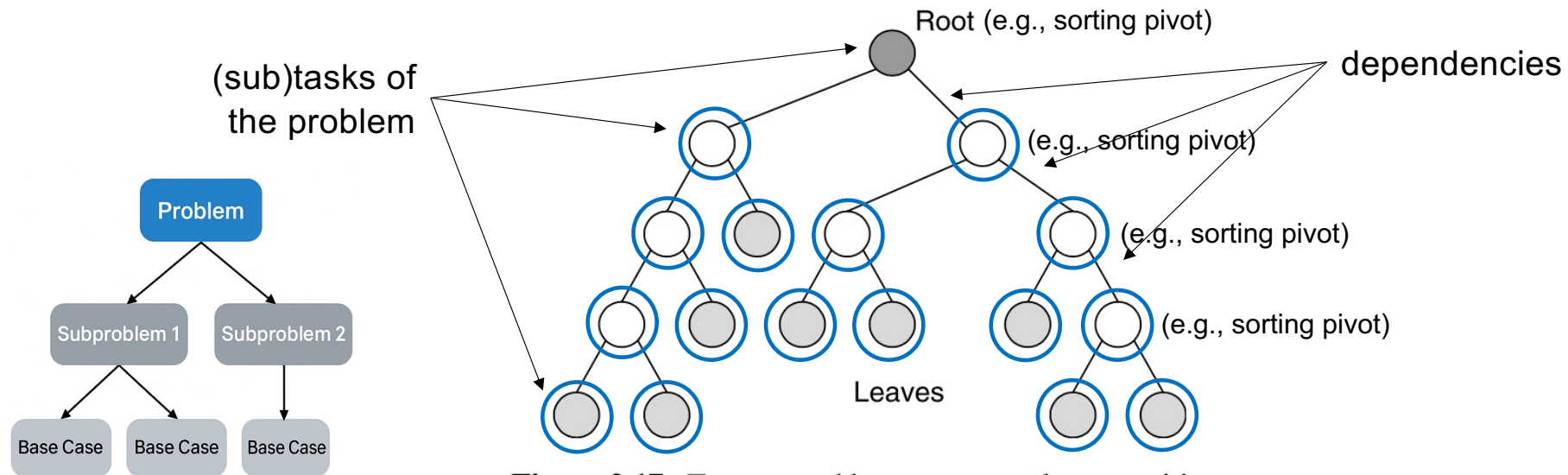


Figure 2.17. Tree created by a **recursive** decomposition.

Type: *recursive decomposition* of a divide-and-conquer problem

- Problem recursively divided into set of independent subproblems; Non-leaf nodes combine the solutions of its subproblems (up to incl. the root)
- Input: program, (max) #concurrency (here **leaves = 9**)
- Output: **partitioned** task sets (here could still be 3)

Applications: functional parallel and/or data parallel problems, such as sorting (quicksort, mergesort), Fibonacci numbers, Mandelbrot set

Parallelization Step 1: Decomposition Type/Example

Exploratory

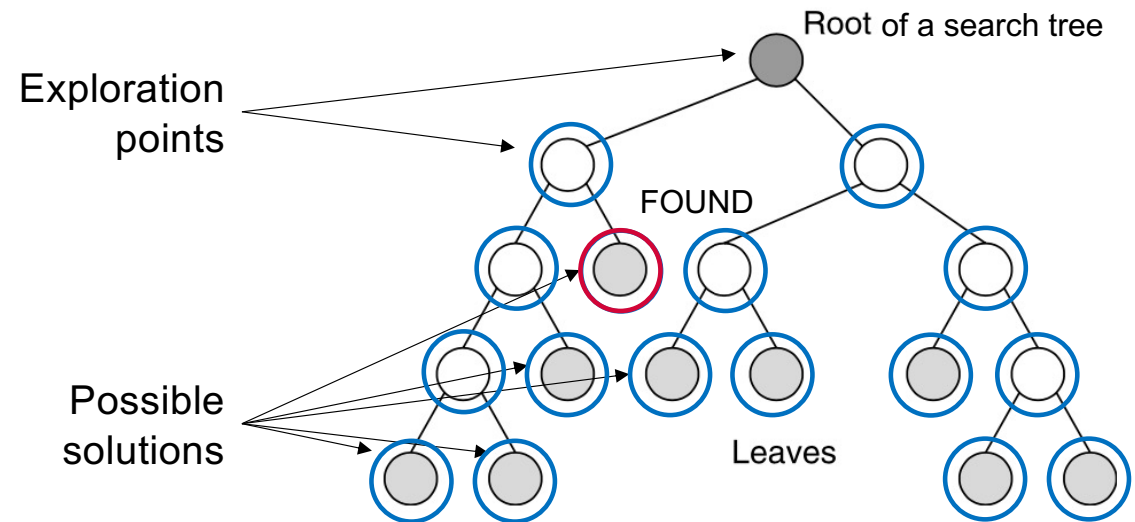
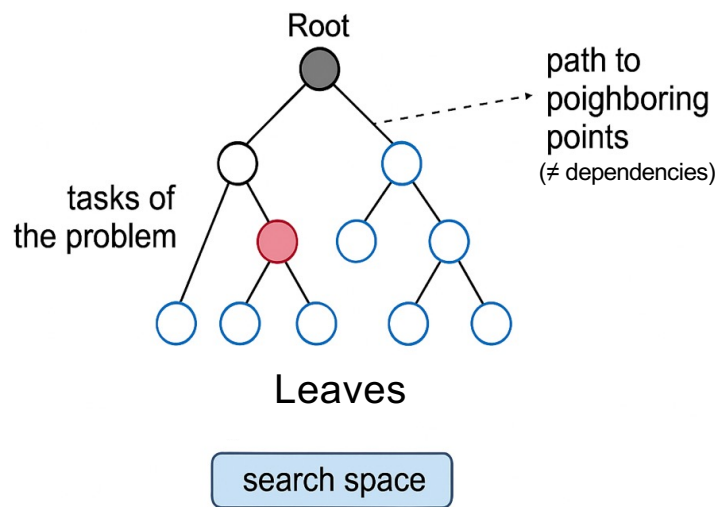


Figure 2.17. Tree created by a **exploratory** decomposition.

Type: *exploratory decomposition* for a search problem

- Search space (= tree) divided (using branch-and-bound) into points in the search space until desired degree of concurrency is met e.g., #leaves = 9
- Computation stops when 1 parallel task found the solution (≠ recursive decomposition)

Applications: optimization, simulation, search problems, such as find minimum, maximum, Monte Carlo methods, pattern matching, game tree exploration, molecular modeling

Parallelization Step 1: Decomposition Type/Example

Speculative

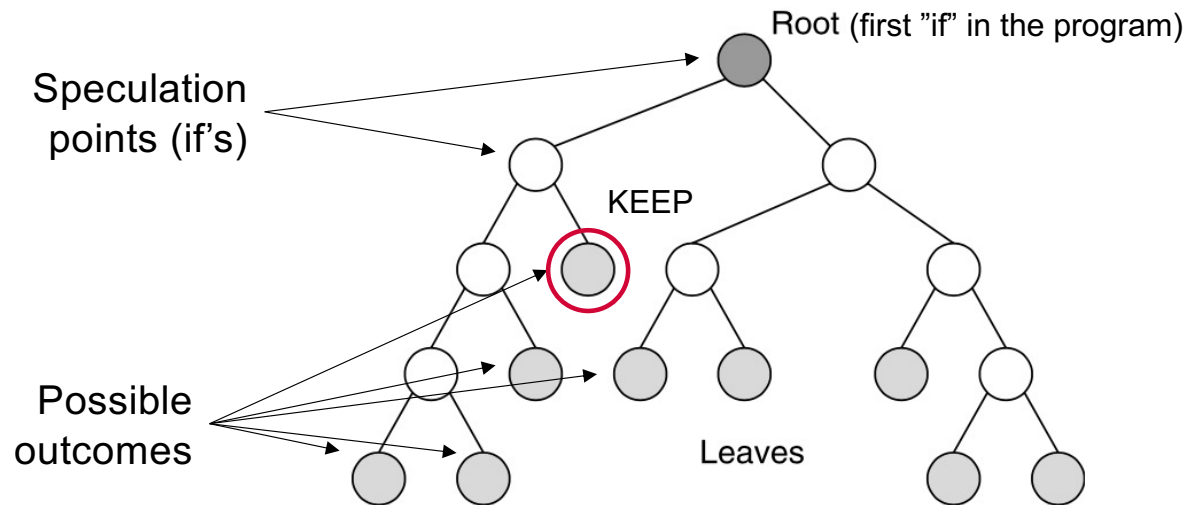
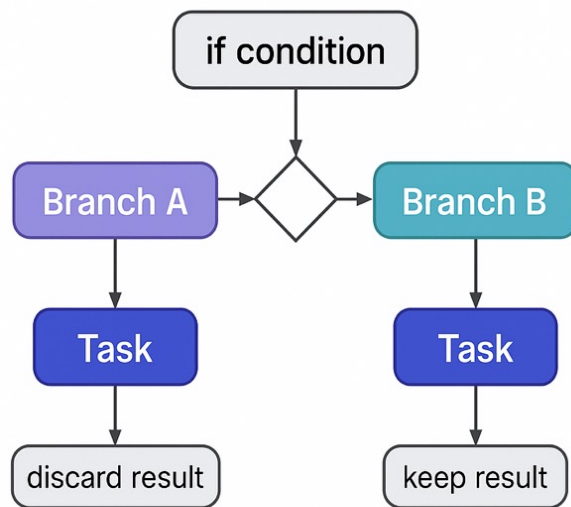


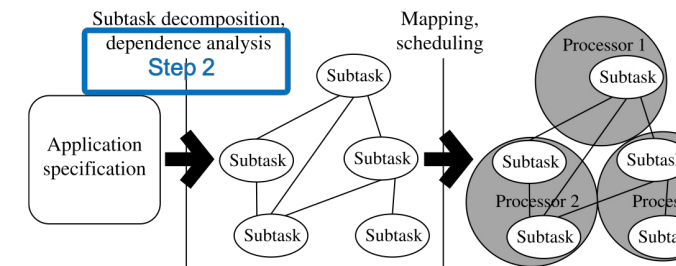
Figure 2.17. Tree created by a **speculative** decomposition.

Type: *speculative decomposition* of a program with if branches

- Choice for a certain computation depends on results from previous step (e.g., "if") but both branches executed speculatively
- Execution is sped up by speculatively executing all (or subset of) branches of the task graph
- Outcome: Keep result only for chosen 'if' branch, discard rest

Applications: compilers, probabilistic problems (predictive execution, query planning), speculative transaction systems (if-then-else speculation, speculative execution, prefetching)

Parallelization Step 2: Dependence Analysis

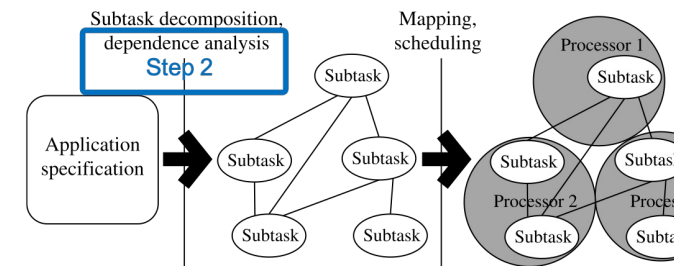


Determining **how** one task depends on another is **critical** to identifying **how much parallelism exists** in a program and **how it can be exploited**.

Control dependencies – describe the *control structure* of a (functional) parallel problem

Impose a *precedence order* on the execution of tasks to preserve **correct** program order

Parallelization Step 2: Dependence Analysis



Determining **how** one task depends on another is **critical** to identifying **how much parallelism exists** in a program and **how it can be exploited**.

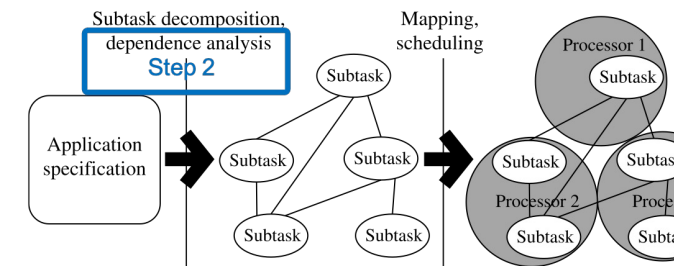
Control dependencies – describe the *control structure* of a (functional) parallel problem

Impose a *precedence order* on the execution of tasks to preserve **correct** program order

Data dependencies – describe the data transfer(s) between tasks performing operations on the data subsets of a (functional or data) **parallel** problem

Flow (true) dependencies – one task *writes*, and another *reads* a variable (also result in a *precedence order* for the execution of tasks, like control dependencies but due to data deps)

Parallelization Step 2: Dependence Analysis



Determining **how** one task depends on another is **critical** to identifying **how much parallelism exists** in a program and **how it can be exploited**.

Control dependencies – describe the *control structure* of a (functional) parallel problem

Impose a *precedence order* on the execution of tasks to preserve **correct** program order

Data dependencies – describe the data transfer(s) between tasks performing operations on the data subsets of a (functional or data) **parallel** problem

Flow (true) dependencies – one task *writes*, and another *reads* a variable (also result in a *precedence order* for the execution of tasks, like control dependencies but due to data deps)

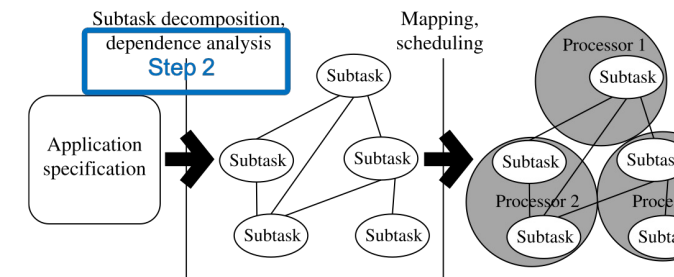
Name dependencies – two tasks use the **same** register/memory location (name) **without any flow of data** between the instructions (no true dependencies) of a (functional or data) **parallel** problem

Anti-dependencies – one task *reads*, and another (over)writes a variable

Output dependencies – both tasks (over)write the same variable

(also result in a *precedence order* for the execution of tasks, like control dependencies but due to name deps)

Parallelization Step 2: Dependence Analysis 1966: Bernstein's Conditions



Assuming statement S_1 and S_2 , S_2 **depends on** S_1 if:

$$[O(S_1) \cap I(S_2)] \cup [I(S_1) \cap O(S_2)] \cup [O(S_1) \cap O(S_2)] \neq \emptyset$$

- $I(S_i)$ is the set of memory locations **read** by S_i
- $O(S_j)$ is the set of memory locations **written** by S_j
- and there is a feasible run-time execution path from S_1 to S_2

Three cases exist to support S_2 **being dependent on** S_1 and one that falsifies the statement

Flow (true) dependencies – one task *writes*, and another *reads* a variable (**ReadAfterWrite**)

$O(S_1) \cap I(S_2) \neq \emptyset$, $S_1 \rightarrow S_2$ and S_1 *writes* something before something is *read* by S_2

Anti-dependencies – one task *reads*, and another (over)*writes* a variable (**WriteAfterRead**)

$I(S_1) \cap O(S_2) \neq \emptyset$, $S_1 \rightarrow S_2$ and S_1 *reads* something before S_2 (over)*writes* it

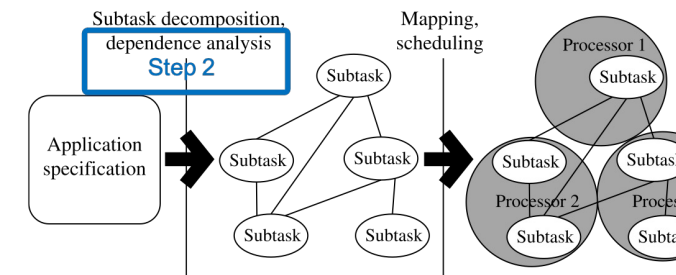
Output dependencies – both tasks (over)*write* the same variable (**WriteAfterWrite**)

$O(S_1) \cap O(S_2) \neq \emptyset$, $S_1 \rightarrow S_2$ and both *write* the same memory location

False dependencies – both tasks read the same location (**ReadAfterRead**)

$I(S_1) \cap I(S_2) \neq \emptyset$, $S_1 \nrightarrow S_2$

Parallelization Step 2: Dependence Analysis



Control dependencies – describe the *control structure* of a (functional) parallel problem

Impose a *precedence order* on the execution of the tasks to preserve correct program order

Example 1: Program for an “if” statement with Control Dependence

1: if **u=0** then

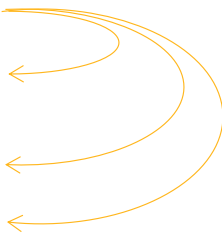
2: v=w

3: else

4: v=w+1

5: x=x-1

6: end if

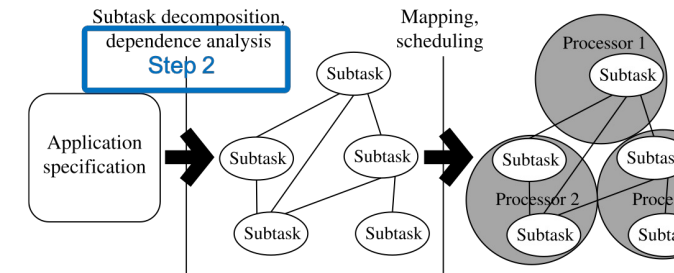


Tasks 2, 4, 5 are **control dependent** on the outcome of task 1 (**if**) (var **u**)

These tasks **cannot be moved** before/after the ‘if’ branch

Tasks 4 and 5 are independent || and can execute **concurrently**, but only after tasks 1 and 3

Parallelization Step 2: Dependence Analysis



Control dependencies – describe the *control structure* of a (functional) parallel problem

Impose a *precedence order* on the execution of the tasks to preserve correct program order

Example 1bis: Program with Control Dependence Transformed to Data Dependence

// techniques for representing and analysis of data dependencies apply/are needed but not shown

1: $b = [u = 0]$ // b is a Boolean variable

|| 2: $v = w$ **when b**
|| 3: $v = w + 1$ **when not b**
4: $x = x - 1$ **when not b**

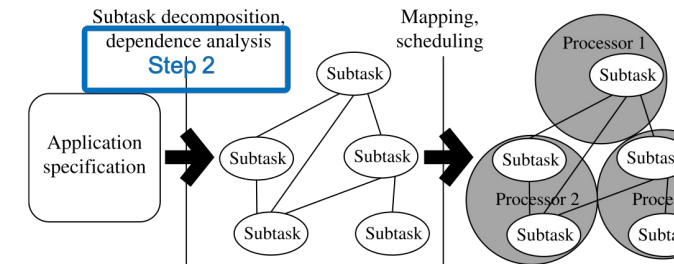


Tasks 2 / 3, 4 only execute when b is true / false, respectively

Tasks 2, 3, 4 are **flow dependent** on task 1 (due to the value of variable b)

Tasks 2, 3, 4 are independent || and can execute **concurrently**, but only after task 1 (precedence order) and once b is known (flow dependence).

Parallelization Step 2: Dependence Analysis



Data dependencies – describe the data transfer(s) between tasks performing operations on the data subsets of a (functional or data) **parallel** problem

Flow (true) dependencies – one task *writes*, and another *reads* a variable (also result in a *precedence order* for the execution of tasks)

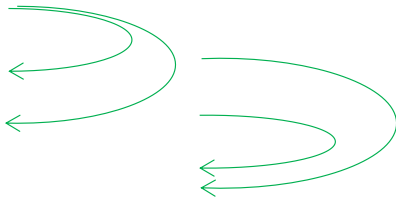
Example 2: Program for $x = a * 7 + (a + 2)$

1: $a = 2$

2: $u = a + 2$

3: $v = a * 7$

4: $x = u + v$

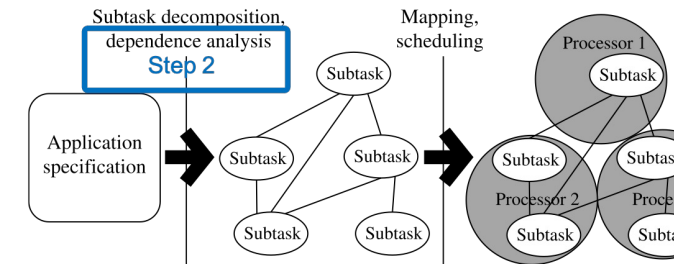


Tasks 2 and 3 are **flow dependent** on task 1 (var a).

Task 4 is **flow dependent** on tasks 2 and 3 (vars u , v).

Tasks 2 and 3 are independent || and can execute **concurrently**, after task 1

Parallelization Step 2: Dependence Analysis



Data dependencies – describe the data transfer(s) between tasks performing operations on the data subsets of a (functional or data) **parallel** problem

Example 3: Program for a Single Loop

for i = 2 to 100 do

 S(i): $A(i) = B(i+2) + 7$
 || T(i): $B(i*2+1) = A(i-1) + C(i+1)$

end for

T(i+1) is **flow dependent** on S(i) due to var A(i)

⇒ **uniform, inter-iteration** dependence

S(3)/S(5)/S(7)..S(99) are **flow dependent** on T(2)/T(3)/T(4)..T(50)

due to the B(i*2+1) output of T and the input B(i+2) of S

⇒ **non-uniform, inter-iteration** dependence

Statements S(i) and T(i) are independent II and **concurrent**.

Execution of First Iterations of the Single Loop

S (2) : $A (2) = B (4) + 7$
 T (2) : $B (5) = A (1) + C (3)$

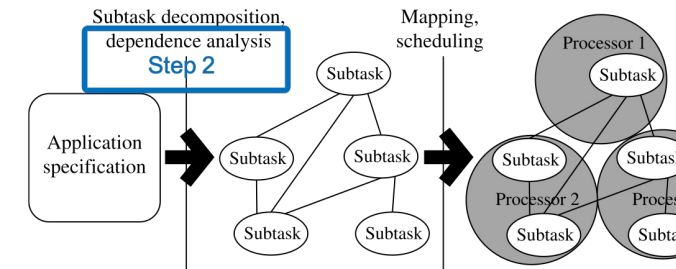
 S (3) : $A (3) = B (5) + 7$
 T (3) : $B (7) = A (2) + C (4)$

 S (4) : $A (4) = B (6) + 7$
 T (4) : $B (9) = A (3) + C (5)$

 S (5) : $A (5) = B (7) + 7$
 T (5) : $B (11) = A (4) + C (6)$

...

Parallelization Step 2: Dependence Analysis



Name dependencies – two tasks use the **same** register/memory location (name) **without any flow of data** between the instructions (no true dependencies) of a (functional or data) **parallel** problem

Anti-dependencies – one task **reads**, and another **writes** a variable

Output dependencies – both tasks **write** that variable

Example 4: Program for $x = a * 7 + (a * 5 + 2)$

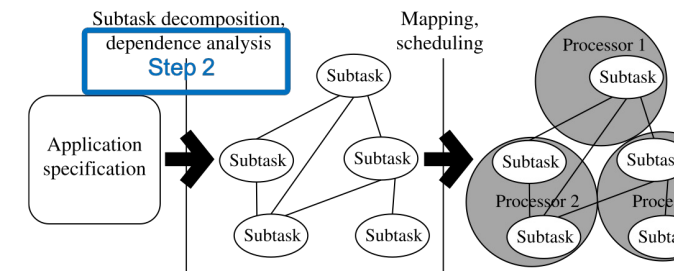
1: $a = 2$
2: $v = a * 5$
3: $u = v + 2$
4: $v = a * 7$
5: $x = u + v$

flow dep. (var a)

Task 4 is **anti-dependent** on task 3 (var v).

flow dep. (vars u, v)

Parallelization Step 2: Dependence Analysis



Name dependencies – two tasks use the **same** register/memory location (name) **without any flow of data** between the instructions (no true dependencies) of a (functional or data) **parallel** problem

Anti-dependencies – one task **reads**, and another **writes** a variable

Output dependencies – both tasks **write** the same variable

Example 4bis: Transformed program with Anti-dependence to **Flow Dependence**

by using additional variables **y** and **w**

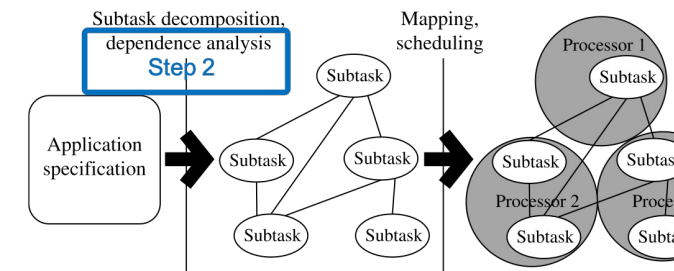
1: a = 2	
2: v = a * 5	flow dep. (var a)
[3: y = v]	flow dep. (var v)
4: u = y + 2	flow dep. (var y)
5: w = a * 7	
6: x = u + w	flow dep. (vars. u , w)

Recall Ex. 4: Program for $x = a * 7 + (a * 5 + 2)$

1: a = 2
2: v = a * 5
3: u = v + 2
4: v = a * 7
5: x = u + v

Task 4 is **anti-dependent** on task 3 (var **v**).

Parallelization Step 2: Dependence Analysis



Name dependencies – two tasks use the **same** register/memory location (name) **without any flow of data** between the instructions (no true dependencies) of a (functional or data) **parallel** problem

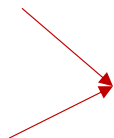
Anti-dependencies – one task *reads*, and another *writes* a variable

Output dependencies – **both** tasks **write** the same variable

Example 5: Output dependence

...
2: **o** = 10

...
4: **o** = 7



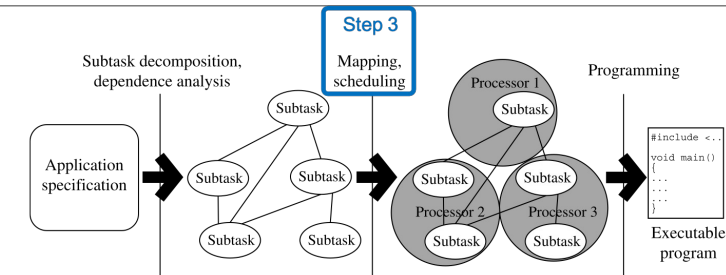
Tasks 2 and 4 are **output dependent** on each other (var **o**).

Parallelization Step 3: Mapping, scheduling

Consists of

Mapping: Spatial assignment – also “**placement**”, refers to placing the subtasks **onto** processing elements, which will execute the subtasks according to the given (or possible) temporal ordering

Answers the question “**Where** (will a subtask be executed)?”



Parallelization Step 3: Mapping, scheduling

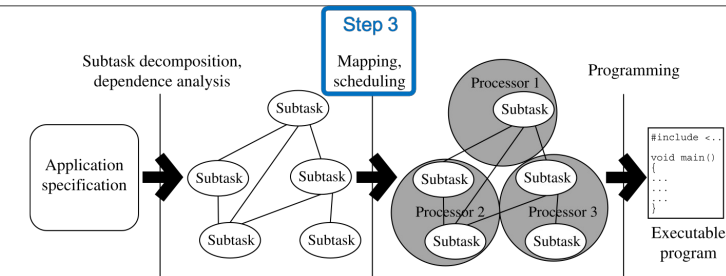
Consists of

Mapping: Spatial assignment – also “placement”, refers to placing the subtasks **onto** processing elements, which will execute the subtasks according to the given (or possible) temporal ordering

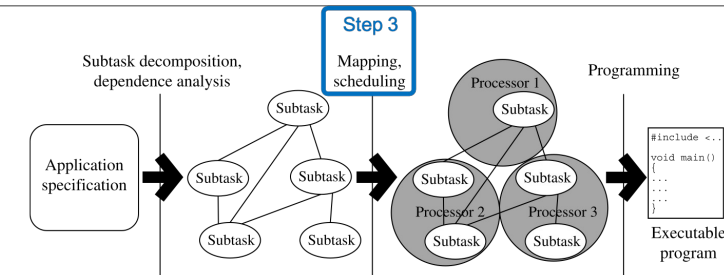
Answers the question “**Where** (will a subtask be executed)?”

Scheduling: Temporal ordering – refers to assigning a *start time* to each subtask

Answers the question “**When** (can a subtask be executed)?”



Parallelization Step 3: Mapping, scheduling



Consists of

Mapping: Spatial assignment – also “placement”, refers to placing the subtasks **onto** processing elements, which will execute the subtasks according to the given (or possible) temporal ordering

Answers the question “**Where** (will a subtask be executed)?”

Scheduling: Temporal ordering – refers to assigning a *start time* to each subtask

Answers the question “**When** (can a subtask be executed)?”

Step 3 can be **Static | Dynamic** – also “**offline**” (before execution) | “**online**” (during execution)

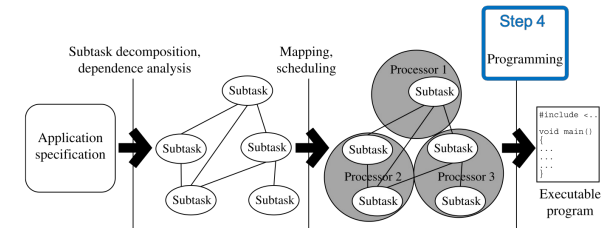
Refers to **the mapping** and **scheduling approaches** for determining “**where?**” (on which processing element) will a subtask be executed and “**when?**” will it execute there

Answers the question “**How is** (a subtask’s) **mapping** in space & **scheduling** in time **performed?**”



Parallelization Step 4: **Parallel** Programming Models and Paradigms

More in Chapters 5.2 (Programming Models) and 5.3 (Parallel Programming)



1. Shared-memory (without threads)
2. **Shared-memory (with threads)**
3. **Distributed-memory / Message-Passing**
4. Data parallel
5. **Hybrid distributed shared-memory**
6. Single Program Multiple Data (SPMD)
7. Multiple Program Multiple Data (MPMD)

Performance is now a programmer's charge. [...] "If programmers want their programs to go faster with each generation, they must make their programs more parallel."

From an Application to its Parallel Execution

A Holistic Perspective

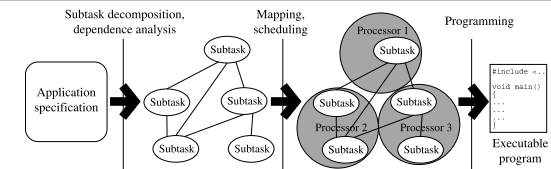
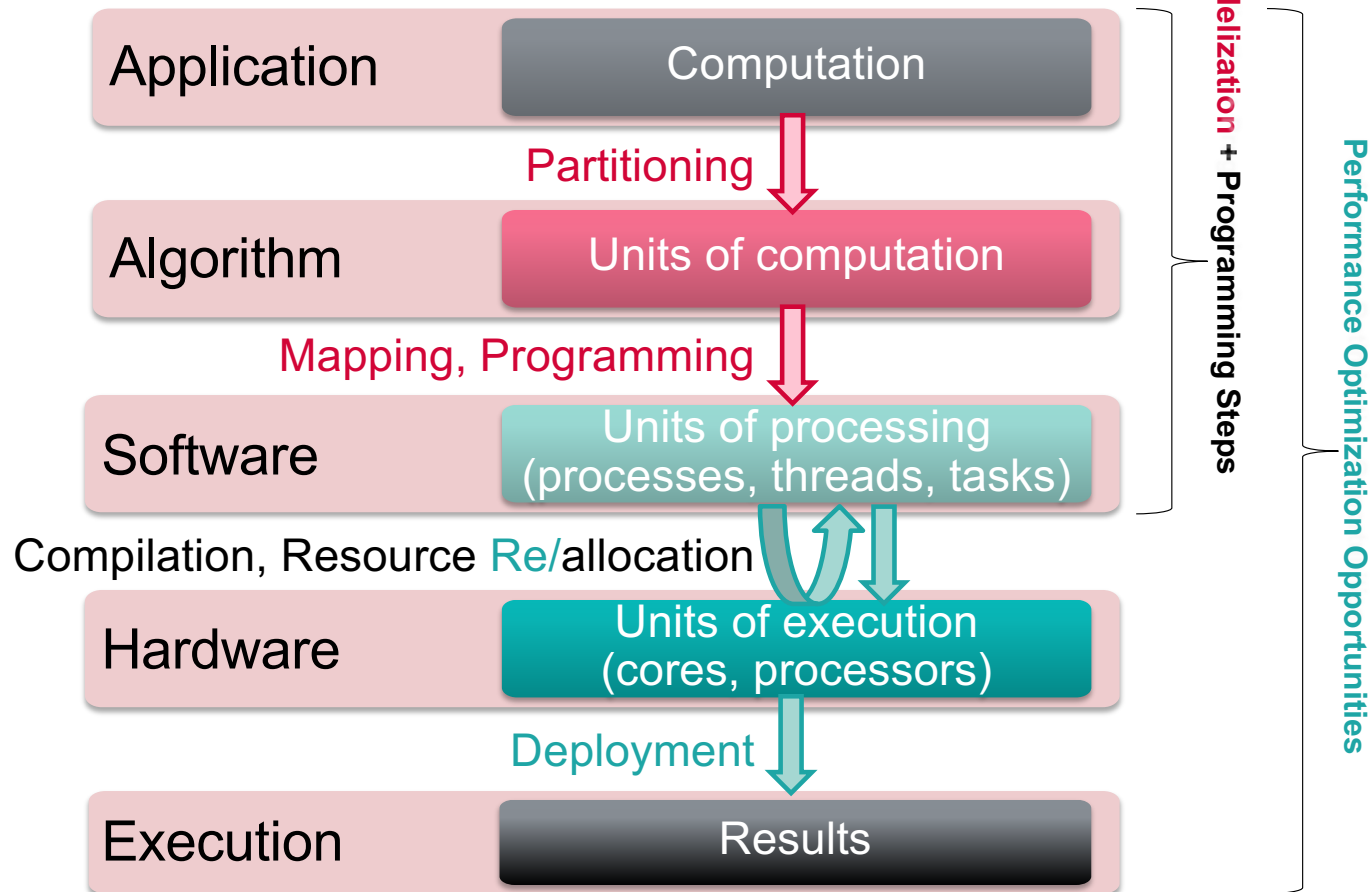
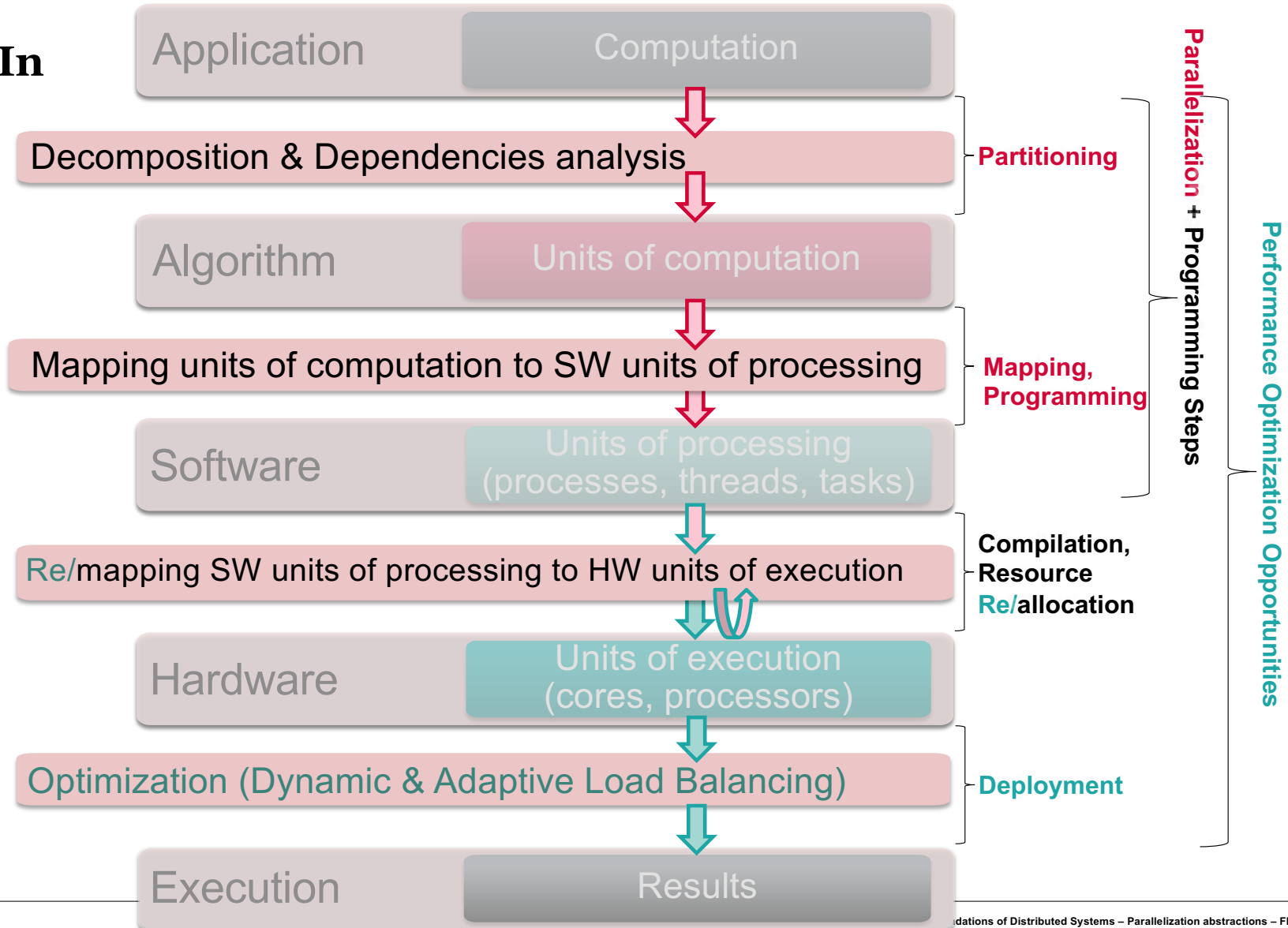


Figure 1.1. Parallel programming—process of parallelization.

Zoom In



What Did We Learn So Far?

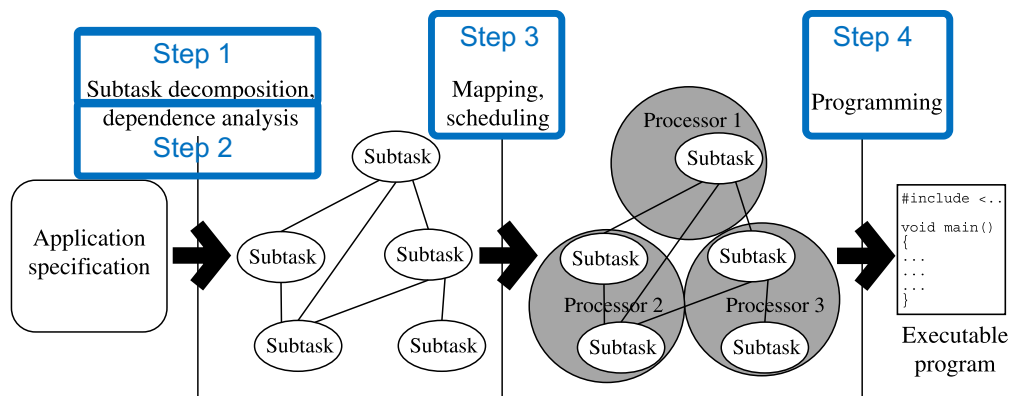


Figure 1.1. Parallel programming—process of parallelization.

a	Parallelization Basic Abstractions
b	Parallelism: Forms, Levels, Machine Models, Types

Summary

- Sequential ≠ Concurrent ≠ Parallel ≠ Distributed
- 4 steps of **parallelization**: decomposition, dependence analysis, mapping, parallel programming
- These occur between Application->Algorithm->Software->Hardware->Execution
- Performance requires **exploiting parallelism** after **exposing concurrency** and is now a programmer's charge